



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

GIC, GIS y GITI

Especificaciones de formato

**Realizado por
(ponente): José Ra. Portillo Fernández**

**Dirigido por
Junta de Centro**

**Departamento
Comisión de Trabajos fin de Grado de la E.T.S.I.I.**

Sevilla, 1 de marzo de 2017 (v.0.91)

Índice general

I Introducción y Conceptos

1	Introducción	2
1.1	Contexto	2
1.2	Motivación	2
1.3	Objetivos	3
1.3.1	Objetivo general	3
1.3.2	Objetivos técnicos	3
1.3.3	Objetivos académicos y personales	3
1.4	Estructura de la memoria	4
2	Preparación del proyecto	5
2.1	Estudio de las tecnologías y toma de decisiones	5
2.1.1	Decisión de frameworks conversacionales	5
2.1.2	Decisión de base de datos	7
2.1.3	Decisión de API de LLM	9
2.2	Infraestructura del sistema	12
2.2.1	Arquitectura del sistema	12
2.2.2	Componentes tecnológicos del sistema	13
2.3	Casos de uso principales	13
2.3.1	Consulta sobre horarios de clases	13
2.3.2	Consulta sobre documentación necesaria para trámites	13
2.3.3	Consulta sobre datos de contacto y tutorías del profesorado	14
2.3.4	Consulta sobre información de una asignatura	15
2.3.5	Consulta sobre normas y documentación de la Universidad	16
3	Conceptos iniciales	17
3.1	Sistema conversacional y componentes de Rasa	17
3.2	Procesamiento del lenguaje natural	17
3.3	Gestión del diálogo	17
3.4	Backend y persistencia de datos	17

II Metodología y Planificación

4	Metodología	19
4.1	SCRUM ajustado en la ejecución	19
4.2	Herramientas	19
5	Planificación	20
5.1	Estructura de Descomposición del Trabajo (EDT)	20
5.2	Diccionario EDT	20
5.2.1	Fase 1 — Inicio	20
5.2.2	Fase 2 — Desarrollo	22
5.2.3	Fase 3 — Cierre	24

5.3	Resumen cuantitativo de la EDT	26
5.4	Backlog del producto	26
5.5	Planificación de sprints	27
5.6	Cronograma	27

III Diseño e Implementación

6	Arquitectura del sistema y decisiones de diseño	29
6.1	Arquitectura general	29
6.1.1	Capa de presentación	29
6.1.2	Capa de procesamiento conversacional	30
6.1.3	Capa de datos	30
6.2	Diagrama de componentes	30
6.3	Diagrama de despliegue	32
6.3.1	Nodos de ejecución	32
6.3.2	Protocolos de comunicación	33
6.4	Decisiones de diseño	33
6.4.1	Text-to-SQL con LLM en lugar de reglas estáticas	33
6.4.2	Gemini como servicio principal con Ollama como alternativa local	34
6.4.3	Embeddings de 2000 dimensiones con HNSW	34
6.4.4	Chunking con solapamiento para preservar contexto	34
6.4.5	Búsqueda vectorial con fallback a búsqueda por palabras clave	35
6.4.6	Fuzzy matching para resolución de nombres	35
6.4.7	Slots conversacionales para memoria de contexto	35
6.4.8	Pipeline NLU de 9 etapas con doble featurización	36
6.4.9	Hash SHA-256 para evitar reprocesamiento de documentos	36
6.5	Patrones arquitectónicos aplicados	37
6.5.1	Arquitectura por capas (<i>Layered Architecture</i>)	37
6.5.2	Patrón <i>Action Server</i> (microservicio de lógica)	37
6.5.3	Patrón <i>Strategy</i> para clientes LLM	37
6.5.4	Patrón <i>Pipeline</i> para procesamiento RAG	37
6.5.5	Patrón <i>Repository</i> para acceso a datos	37
7	Marco tecnológico	39
7.1	Tecnologías de procesamiento conversacional	39
7.1.1	Rasa Framework	39
7.1.2	Procesamiento de Lenguaje Natural	40
7.2	Tecnologías de Inteligencia Artificial	40
7.2.1	Ollama y modelos de lenguaje grandes (LLMs)	40
7.2.2	Embeddings y búsqueda semántica	41
7.3	Tecnologías de base de datos	41
7.3.1	PostgreSQL	41
7.3.2	Supabase	42
7.3.3	pgvector	42
7.4	Tecnologías de frontend	42
7.4.1	Node.js	42
7.4.2	Framework web	42
7.5	Herramientas de desarrollo	43
7.5.1	Control de versiones	43
7.5.2	Gestión de dependencias	43
7.5.3	Variables de entorno	43
7.6	Consideraciones de seguridad	43

7.6.1	Protección de datos	43
7.6.2	Validación de entrada	43
7.6.3	Ejecución local de modelos	44
7.7	Stack tecnológico completo	44
8	Estructura del proyecto	45
8.1	Visión general del repositorio	45
8.2	Directorio raíz	46
8.3	Directorio <code>actions/</code>	46
8.3.1	Organización de los módulos	46
8.3.2	Flujo de ejecución de una acción	47
8.4	Directorio <code>data/</code>	47
8.4.1	Subdirectorio <code>nlu/</code>	47
8.4.2	Ficheros de flujo conversacional	48
8.5	Directorio <code>frontend/</code>	48
8.6	Directorio <code>data/</code> de Rasa vs. docs/	48
8.7	Directorio <code>tests/</code>	48
8.8	Separación de responsabilidades	49
9	Estructura de la base de datos	50
9.1	Diseño conceptual	50
9.1.1	Principios de diseño	50
9.2	Modelo entidad-relación	50
9.3	Entidades principales	51
9.3.1	Universidades	51
9.3.2	Centros	51
9.3.3	Departamentos	51
9.3.4	Titulaciones	51
9.3.5	Asignaturas	51
9.3.6	Profesores	53
9.3.7	Tutorías	53
9.3.8	Horarios	53
9.4	Tablas de vectorización (RAG)	53
9.4.1	Planes docentes	54
9.4.2	Chunks de planes docentes	54
9.5	Tablas de trámites administrativos	56
9.5.1	Categorías de trámites	56
9.5.2	Trámites	56
9.6	Relaciones entre entidades	56
9.7	Índices y optimización	56
9.8	Integridad referencial	56
9.9	Estrategia de datos de prueba	57
	Referencias	58

Índice de figuras

5.1	Estructura de Descomposición del Trabajo (EDT) — fases y paquetes de trabajo.	21
6.1	Diagrama de componentes del sistema Linceus.	31
6.2	Diagrama de despliegue del sistema Linceus en DigitalOcean (Ubuntu 24.04 LTS) con el pipeline de despliegue automatizado vía GitHub Actions.	38

Índice de tablas

2.1	Comparación de frameworks conversacionales	8
2.2	Comparación de bases de datos	9
2.3	Comparación de APIs de LLM	12
2.4	Componentes tecnológicos del sistema	13
5.1	Diccionario EDT — 1.1: Gestión del proyecto.	21
5.2	Diccionario EDT — 1.2: Investigación tecnológica.	22
5.3	Diccionario EDT — 1.3: Diseño de la arquitectura.	22
5.4	Diccionario EDT — 2.1: Épica Asignaturas.	23
5.5	Diccionario EDT — 2.2: Épica Profesores.	23
5.6	Diccionario EDT — 2.3: Épica Horarios.	24
5.7	Diccionario EDT — 2.4: Pipeline RAG.	24
5.8	Diccionario EDT — 2.5: Panel admin y frontend.	25
5.9	Diccionario EDT — 3.1: Validación y testing.	25
5.10	Diccionario EDT — 3.2: Piloto con usuarios.	25
5.11	Diccionario EDT — 3.3: Despliegue en producción.	26
5.12	Diccionario EDT — 3.4: Memoria y defensa.	26
5.13	Resumen cuantitativo de la EDT por fase y paquete de trabajo.	27
5.14	Priorización del backlog dentro de la fase de Desarrollo.	27
6.1	Protocolos y puertos de comunicación entre nodos del sistema.	33
7.1	Stack tecnológico completo del proyecto LinceUS Assistant.	44
9.1	Estructura de la tabla UNIVERSIDADES	51
9.2	Estructura de la tabla CENTROS	51
9.3	Estructura de la tabla DEPARTAMENTOS	52
9.4	Estructura de la tabla TITULACIONES	52
9.5	Estructura de la tabla ASIGNATURAS	52
9.6	Estructura de la tabla PROFESORES	53
9.7	Estructura de la tabla TUTORIAS	53
9.8	Estructura de la tabla HORARIOS	54
9.9	Estructura de la tabla PLANES_DOCENTES	54
9.10	Estructura de la tabla PLANES_DOCENTES_CHUNKS	55
9.11	Estructura de la tabla CATEGORIAS_TRAMITES	55
9.12	Estructura de la tabla TRAMITES	55

Índice de código

Parte I

Introducción y Conceptos

Introducción

Este proyecto tiene como finalidad el diseño y desarrollo de un chatbot inteligente orientado a facilitar el acceso a información académica y administrativa en la Universidad de Sevilla, mediante técnicas de procesamiento del lenguaje natural y recuperación aumentada por generación (RAG).

1.1– Contexto

En la actualidad, el alumnado de la Universidad de Sevilla se enfrenta a una notable dispersión informativa. La información académica y administrativa se encuentra repartida entre múltiples plataformas y páginas web, cada una gestionada de forma independiente por distintas facultades, servicios y departamentos. Esta fragmentación dificulta el acceso eficiente a datos esenciales como horarios, contacto con profesorado, requisitos para trámites o metodologías de asignaturas.

Además, muchos contenidos relevantes no se presentan de forma unificada ni están estructurados para su consulta directa, sino que requieren navegar por documentos PDF (como proyectos docentes), correos informativos, páginas institucionales específicas u otros portales como Sevius. Esta situación resulta especialmente compleja para estudiantes de nuevo ingreso o internacionales, que no siempre conocen el funcionamiento interno del entorno universitario y, en muchos casos, deben enfrentarse también a la barrera del idioma, lo que dificulta aún más la comprensión de la información disponible.

La ausencia de un punto de acceso único, interactivo y orientado al usuario final pone de manifiesto la necesidad de una herramienta conversacional que permita acceder a la información de manera contextual, simplificada y guiada. En este sentido, un chatbot capaz de interpretar consultas abiertas en lenguaje natural y conectarse con diferentes fuentes de información representa una solución innovadora, alineada con los principios de accesibilidad, automatización y modernización de los servicios universitarios.

1.2– Motivación

La motivación principal de este proyecto surge de la necesidad de centralizar el acceso a información académica dispersa y mejorar la experiencia del estudiante en su interacción con los sistemas universitarios. Los estudiantes, especialmente aquellos de nuevo ingreso o de programas de intercambio, se enfrentan con frecuencia a dificultades para localizar información básica como horarios de clase, datos de contacto del profesorado o documentación necesaria para realizar trámites administrativos.

Esta problemática no solo genera frustración y pérdida de tiempo, sino que también puede afectar negativamente al rendimiento académico y a la integración del estudiante en la comunidad

universitaria. La dispersión de la información en múltiples fuentes web, documentos PDF y portales específicos dificulta la autonomía informativa del alumnado, que debe dedicar un esfuerzo significativo a buscar respuestas que deberían estar disponibles de forma inmediata.

Por tanto, el desarrollo de un asistente conversacional capaz de responder de forma rápida, precisa y en lenguaje natural constituye una mejora sustancial en la calidad del servicio universitario, promoviendo la autonomía del estudiante y reduciendo la carga administrativa de los servicios de atención.

1.3– Objetivos

1.3.1. Objetivo general

El objetivo principal de este proyecto es diseñar y desarrollar un chatbot inteligente que proporcione a los estudiantes de la Universidad de Sevilla una vía centralizada, accesible y eficiente para consultar información académica y administrativa relevante. La solución pretende integrar capacidades de comprensión del lenguaje natural mediante modelos de Inteligencia Artificial Generativa, junto con búsquedas en bases de datos estructuradas y vectorizadas, permitiendo resolver dudas frecuentes relacionadas con horarios, profesorado, documentación, normativas o localización de espacios universitarios. El asistente está especialmente orientado a facilitar la autonomía informativa de estudiantes de nuevo ingreso, internacionales o aquellos menos familiarizados con los procedimientos institucionales de la Universidad de Sevilla.

1.3.2. Objetivos técnicos

Los objetivos técnicos específicos del proyecto incluyen:

- Implementar un sistema conversacional basado en Rasa que permita gestionar flujos de diálogo complejos y contextuales.
- Integrar la API de Gemini para la detección de intenciones y generación de respuestas enriquecidas mediante lenguaje natural.
- Diseñar y desarrollar una base de datos híbrida en Supabase que combine información estructurada (horarios, profesorado, trámites) con embeddings vectoriales para búsquedas semánticas.
- Desarrollar acciones personalizadas en Python que permitan consultar datos estructurados y realizar búsquedas por similitud en documentación académica.
- Implementar un frontend web en React que ofrezca una interfaz intuitiva y accesible para la interacción con el chatbot.
- Garantizar el cumplimiento de normativas de privacidad (RGPD) mediante despliegue local y control de acceso a datos sensibles.

1.3.3. Objetivos académicos y personales

Desde el punto de vista académico y personal, este proyecto permite:

- Aplicar conocimientos adquiridos durante el Grado en Ingeniería del Software en un proyecto real de desarrollo de software.
- Adquirir experiencia práctica en el diseño de sistemas conversacionales y en la integración de modelos de inteligencia artificial generativa.

- Profundizar en el uso de técnicas de Retrieval Augmented Generation (RAG) y bases de datos vectoriales.
- Desarrollar competencias en metodologías ágiles aplicadas a proyectos individuales.
- Contribuir a la mejora de los servicios universitarios mediante una solución tecnológica innovadora y accesible.
- Demostrar la capacidad de diseñar, implementar y documentar un sistema completo de forma autónoma.

1.4– Estructura de la memoria

Esta memoria se organiza en dos partes principales:

Parte I: Introducción y Conceptos, donde se contextualiza el proyecto, se justifica su necesidad, se definen los objetivos y se describen los conceptos técnicos fundamentales relacionados con sistemas conversacionales, procesamiento del lenguaje natural y arquitectura del sistema.

Parte II: Metodología y Planificación, que detalla la metodología de desarrollo empleada (SCRUM adaptado), las herramientas utilizadas y la planificación temporal del proyecto, incluyendo el backlog de tareas y la distribución en sprints.

Preparación del proyecto

En esta fase previa se llevó a cabo un análisis exhaustivo de las tecnologías disponibles para construir el sistema conversacional, evaluando alternativas en función de criterios como coste, facilidad de integración, capacidad de despliegue local y compatibilidad con técnicas de RAG. Asimismo, se definió la arquitectura técnica del sistema y se identificaron los casos de uso principales que guiarían el desarrollo del proyecto.

2.1– Estudio de las tecnologías y toma de decisiones

2.1.1. Decisión de frameworks conversacionales

Frameworks analizados

Se han considerado distintos frameworks de chatbot **open source** que cumplen con los requisitos de despliegue local, integración en Python y soporte para RAG. Los principales son:

- **Rasa**, centrado en Python y con gran control de flujo.
- **Botpress (OSS)**, que destaca por su editor visual de flujos.
- **DeepPavlov**, más orientado a NLP avanzado con módulos reutilizables.
- **OpenDialog**, centrado en diseño visual y contextos conversacionales.
- **Tock** y **Botonic**, opciones alternativas con características interesantes, aunque con menor adopción o diferente enfoque tecnológico.

Comparación detallada

Flujo conversacional El flujo conversacional se refiere a la capacidad del framework para **definir, controlar y gestionar el diálogo entre el usuario y el sistema**. Incluye cómo se modelan las intenciones, las respuestas, los contextos y la memoria de la conversación, así como el grado de flexibilidad para manejar **diálogos no lineales, interrupciones o escenarios complejos**. Un buen manejo del flujo conversacional permite que el chatbot no solo siga guiones rígidos, sino que pueda adaptarse dinámicamente a las necesidades del usuario, manteniendo coherencia y contexto a lo largo de la interacción.

- **Rasa** ofrece máximo control mediante historias y reglas, permitiendo modelar diálogos complejos y contextuales.

- **Botpress** facilita el diseño con nodos visuales, aunque está más orientado a flujos rígidos y predefinidos.
- **DeepPavlov** permite construir agentes multi-skill, pero requiere definir manualmente los pipelines.
- **OpenDialog** sobresale en el modelado de escenarios y contextos, aunque su dependencia de PHP limita la integración con Python.
- **Tock/Botonic** permiten definir historias o rutas conversacionales, pero con menor madurez que Rasa.

Testing La capacidad de testing en un framework de chatbots hace referencia a las **herramientas disponibles para validar la calidad y el correcto funcionamiento del agente conversacional**. Esto incluye desde pruebas unitarias (validación de intenciones, entidades y respuestas individuales) hasta pruebas end-to-end (que simulan conversaciones completas para comprobar flujos y contextos). Un buen soporte de testing permite **automatizar la detección de errores, reducir el riesgo de alucinaciones o respuestas incoherentes y asegurar la estabilidad** del sistema a medida que evoluciona. Cuanto más integrado esté el testing en el framework, más fácil será mantener y escalar el chatbot con garantías de calidad.

- **Rasa** incorpora herramientas para pruebas unitarias y end-to-end, automatizando la validación de intenciones y flujos.
- **Botpress** ofrece principalmente un simulador manual, sin suite de pruebas automatizadas robusta.
- **DeepPavlov** no incluye framework específico, por lo que las pruebas deben hacerse con scripts en Python.
- **OpenDialog** y **Botonic** dependen casi exclusivamente de pruebas manuales.

Compatibilidad con RAG y embeddings La compatibilidad con **RAG (Retrieval Augmented Generation)** y el uso de **embeddings** mide hasta qué punto un framework permite integrar modelos de lenguaje con bases de datos vectoriales para mejorar las respuestas. Esta característica es clave cuando se trabaja con **información documental extensa o cambiante**. Un buen soporte en este ámbito asegura que el chatbot pueda **ofrecer respuestas actualizadas, precisas y fundamentadas en datos externos**, reduciendo el riesgo de respuestas inventadas (alucinaciones).

- **Rasa** se integra de forma natural con bases vectoriales (pgvector, Qdrant, FAISS), permitiendo retrieval augmented generation.
- **Botpress** dispone de un módulo de *Knowledge Base* que admite cargar documentos, aunque menos flexible en personalización.
- **DeepPavlov** destaca en Q&A y FAQ matching, aunque el RAG debe construirse manualmente.
- **OpenDialog** y **Botonic** no tienen soporte nativo, siendo necesaria integración externa.

Despliegue local y privacidad El despliegue local y la gestión de la privacidad se refieren a la **posibilidad de ejecutar el framework en entornos controlados (on-premise o en servidores propios)** sin depender necesariamente de servicios en la nube. Esta característica es importante cuando se manejan **datos sensibles o confidenciales**, ya que permite aplicar políticas de seguridad personalizadas y cumplir con normativas como el RGPD. Además, la facilidad de despliegue (con Docker, instaladores nativos o stacks ligeros) influye directamente en la **rapidez de la puesta en marcha y en la capacidad de mantener el control total sobre la información del usuario**.

- **Rasa** y **Botpress** son fácilmente desplegables en local mediante Docker o instalación directa.
- **DeepPavlov** funciona como librería Python o servicio REST.
- **OpenDialog** requiere un stack PHP más pesado.
- **Tock** es estable en entornos locales, mientras que Botonic se orienta a proyectos en Node/React.

Comunidad y documentación La comunidad y la documentación de un framework son indicadores clave de su madurez, accesibilidad y sostenibilidad a largo plazo. Una comunidad activa ofrece foros, repositorios y soporte colaborativo que facilitan la resolución de problemas y el intercambio de buenas prácticas. Al mismo tiempo, una documentación clara, actualizada y completa reduce la curva de aprendizaje y permite implementar soluciones más rápido.

- **Rasa** tiene la comunidad más amplia y documentación extensa.
- **Botpress** dispone de foros y Discord activos, pero con menor alcance.
- **DeepPavlov** cuenta con una comunidad académica sólida, aunque más técnica.
- **OpenDialog** y **Botonic** tienen comunidades pequeñas y más limitadas.

Decisión final

El framework seleccionado es **Rasa**, ya que combina:

- Máximo control sobre el flujo conversacional.
- Capacidades de testing integradas y automatizables.
- Soporte probado para RAG y bases vectoriales.
- Despliegue local sencillo y seguro, compatible con RGPD.
- Amplia comunidad y documentación, lo que garantiza sostenibilidad del proyecto.

Tabla de comparación detallada de frameworks

2.1.2. Decisión de base de datos

Bases de Datos analizadas

Se revisaron diferentes opciones open source para almacenar tanto información estructurada como embeddings:

- **Supabase (Postgres + pgvector).**
- **PostgreSQL autogestionado con pgvector.**
- **Appwrite, PocketBase, Directus, Hasura.**
- Motores vectoriales dedicados: **Qdrant, Weaviate, Milvus, FAISS.**

Característica / Framework	Rasa	Botpress	DeepPavlov	OpenDialog	Tock	Botonic
Flujo conversacional	***	**	**	**	*	*
Testing	***	*	*	*	*	*
Compatibilidad con RAG y embeddings	***	**	**	*	*	*
Despliegue local y privacidad	***	***	**	*	**	**
Comunidad y documentación	***	**	**	*	*	*
Resultado global	* 15	* 10	* 9	* 6	* 6	* 6

Cuadro 2.1: Comparación de frameworks conversacionales

Comparación detallada

Modelo de datos El modelo de datos define cómo se organiza, almacena y consulta la información dentro de una base de datos o motor de persistencia. Dependiendo de la tecnología, este modelo puede ser relacional (tablas y relaciones SQL), orientado a documentos (estructuras tipo JSON), o especializado en vectores (espacios multidimensionales para embeddings). La elección del modelo es clave para garantizar la eficiencia, flexibilidad y escalabilidad del sistema, ya que influye directamente en cómo se estructuran los datos de los usuarios, los recursos académicos y los embeddings utilizados en búsquedas semánticas o RAG.

Búsqueda vectorial

- **Supabase/Postgres** soportan pgvector, adecuado para corpus medianos.
- **Qdrant, Weaviate y Milvus** son más eficientes en grandes volúmenes de vectores.
- **FAISS** funciona muy bien embebido en Python, pero no es una base de datos como tal.
- El resto (Appwrite, PocketBase, Directus, Hasura) carecen de soporte nativo y requieren integraciones manuales.

Despliegue local y facilidad El despliegue local y la facilidad de instalación miden la **complejidad técnica y los recursos necesarios para poner en marcha una base de datos en un entorno controlado**. Algunas soluciones ofrecen entornos ligeros y rápidos de ejecutar (como binarios únicos o imágenes Docker sencillas), mientras que otras requieren stacks más pesados y múltiples dependencias. Esta característica es importante porque impacta directamente en la **rapidez de la configuración inicial, la curva de aprendizaje y los costes de mantenimiento**.

- **Supabase y Postgres** se despliegan fácilmente con Docker.
- **PocketBase** es el más liviano (un solo binario).
- **Appwrite y Directus** requieren más dependencias.
- **Qdrant/Weaviate/Milvus** añaden complejidad, aunque escalables.

Seguridad y RGPD La seguridad y el cumplimiento normativo son aspectos fundamentales en la gestión de datos, especialmente cuando se trabaja con información sensible de estudiantes o usuarios. Esta característica evalúa los **mecanismos nativos de autenticación, autorización y control de acceso** que ofrece cada base de datos, así como la posibilidad de aplicar políticas de privacidad. Tecnologías que incluyen funciones como **Row-Level Security, roles definidos o autenticación integrada** facilitan el cumplimiento legal y reducen riesgos. En cambio, los motores que carecen de seguridad propia obligan a implementar medidas adicionales en la capa de aplicación, aumentando la complejidad y la responsabilidad del equipo de desarrollo.

- **Supabase** ofrece Row-Level Security (RLS) y autenticación JWT.
- **Postgres** tradicional depende de roles SQL.
- **Appwrite/Directus/Hasura** ofrecen mecanismos de auth integrados.
- **Motores vectoriales** carecen de autenticación propia, requiriendo protección adicional en la aplicación.

Comunidad y documentación La comunidad y la documentación determinan el **nivel de soporte, recursos y acompañamiento disponibles para desarrolladores**. Una base de datos con una comunidad amplia y activa facilita la resolución de problemas, la existencia de librerías y la mejora continua del sistema. Al mismo tiempo, una documentación clara, actualizada y con ejemplos prácticos acelera la adopción de la tecnología y reduce la curva de aprendizaje.

- **Postgres** es el más maduro y con mayor soporte.
- **Supabase** ha crecido mucho, con amplia documentación y ejemplos de IA.
- **Qdrant y Weaviate** cuentan con comunidades activas en el ámbito de vectores.
- **Appwrite, PocketBase, Directus** tienen comunidades menores pero en expansión.

Decisión final

La base de datos seleccionada es **Supabase (Postgres + pgvector)**, porque:

- Permite unificar datos estructurados y vectores en una sola plataforma.
- Ofrece autenticación, reglas de seguridad a nivel de fila y APIs listas para consumir desde Python y React.
- Facilita el despliegue local sin costes de licencias.
- Su comunidad y documentación reducen la curva de aprendizaje.
- Cumple los requisitos de RAG con un volumen de datos manejable para el caso académico.

Tabla de comparación de bases de datos

Característica / BD	Supabase	Postgres	Appwrite	PocketBase	Directus	Hasura	Qdrant	Weaviate	Milvus	FAISS
Modelo de datos	***	***	**	**	**	**	**	**	**	**
Búsqueda vectorial	**	**	*	*	*	*	***	***	***	**
Despliegue local y facilidad	***	***	**	***	**	**	**	**	**	**
Seguridad y RGPD	***	**	**	*	**	**	*	*	*	*
Comunidad y documentación	***	***	**	**	**	**	**	**	**	**
Resultado global	★ 14	★ 13	★ 9	★ 9	★ 9	★ 9	★ 10	★ 10	★ 10	★ 9

Cuadro 2.2: Comparación de bases de datos

2.1.3. Decisión de API de LLM

APIs analizadas

Se han considerado diferentes opciones de APIs de modelos de lenguaje de gran tamaño (LLMs) open source o con planes gratuitos/educativos:

- **Gemini (Google AI / DeepMind)**

- **OpenAI GPT (GPT-4o, GPT-4o mini)**
- **DeepSeek**
- **Hugging Face Inference API**
- **Cohere y Anthropic (Claude)**

Comparación detallada

Coste y accesibilidad El coste y la accesibilidad hacen referencia a la **disponibilidad económica y facilidad de uso de los modelos de lenguaje (LLMs)**. En un contexto académico, donde no suele existir presupuesto, es esencial contar con opciones que permitan **experimentar gratuitamente o con bajos costes iniciales**. Además del precio, influyen factores como los límites de uso en los planes gratuitos, la velocidad de respuesta, la estabilidad del servicio y la facilidad de registro o integración. Esta característica resulta clave para asegurar que el proyecto pueda desarrollarse de forma **sostenible y realista**, sin que los costes de las llamadas a la API se conviertan en una barrera.

- Gemini ofrece un plan gratuito con un número considerable de llamadas mensuales, lo que permite experimentar y probar sin coste, factor clave en un TFG sin presupuesto.
- OpenAI GPT tiene un coste asociado desde la primera llamada más allá de GPT-3.5, lo que lo hace menos viable económicamente.
- DeepSeek tiene una API gratuita, pero con menos garantías de estabilidad y soporte.
- Hugging Face API ofrece acceso a modelos open source, pero el plan gratuito tiene fuertes limitaciones de velocidad y peticiones.
- Claude (Anthropic) y Cohere requieren suscripción desde el inicio, menos atractivas en un contexto universitario.

Calidad y rendimiento La calidad y el rendimiento de un modelo de lenguaje se refieren a su capacidad para **comprender instrucciones, generar texto coherente y relevante, y mantener el contexto a lo largo de la conversación**. Estos aspectos suelen evaluarse mediante **pruebas comparativas de comprensión, razonamiento y generación en varios idiomas**. Además, influyen factores prácticos como la **velocidad de respuesta, la estabilidad del modelo y su habilidad para manejar entradas extensas sin pérdida de coherencia**. Una mayor calidad en este ámbito se traduce en **respuestas más útiles, precisas y naturales**, lo que resulta fundamental para un chatbot académico que debe manejar información variada con fiabilidad.

Estudios comparativos muestran que Gemini compite con GPT-4 en comprensión y generación de texto, especialmente en multilingüe y razonamiento.

- GPT sigue siendo referencia en benchmarks de calidad, pero con coste.
- DeepSeek ha demostrado buen rendimiento en ciertas tareas de razonamiento matemático, aunque en comprensión semántica y contexto extenso suele quedar por debajo de Gemini y GPT.
- Hugging Face API depende mucho del modelo seleccionado (ej. LLaMA, Mistral, Falcon), pero requieren más trabajo de fine-tuning y optimización para igualar a Gemini/GPT en calidad.

Integración y soporte técnico La integración y el soporte técnico se refieren a la **facilidad con la que un LLM puede incorporarse en el stack tecnológico existente** y al nivel de recursos disponibles para resolver problemas durante el desarrollo. Factores como la existencia de **SDKs oficiales, librerías en distintos lenguajes, ejemplos prácticos y documentación clara** determinan la rapidez de adopción. Además, contar con un ecosistema sólido y acuerdos institucionales (como convenios académicos) puede simplificar la integración y asegurar **mayor estabilidad a largo plazo**. Esta característica es clave, ya que permite centrar el esfuerzo en el desarrollo del chatbot sin tener que invertir demasiado tiempo en resolver barreras técnicas de conexión con el modelo.

- Gemini ofrece SDKs y buena integración en Python/JavaScript, lo que encaja perfectamente con tu stack (Python + React).
- GPT también tiene SDKs muy maduros, aunque con la barrera de costes.
- DeepSeek todavía no cuenta con ecosistema sólido.
- Hugging Face API es flexible pero requiere seleccionar y mantener modelos específicos.
- Gemini además está alineado con acuerdos académicos, como el que mantiene la **Universidad de Sevilla con Google** en diferentes ámbitos, lo que puede facilitar futuras integraciones oficiales del chatbot.

Privacidad y cumplimiento (RGPD) La privacidad y el cumplimiento normativo son esenciales al utilizar modelos de lenguaje que procesan datos de usuarios. Esta característica evalúa el **nivel de control sobre dónde y cómo se almacenan los datos**, si el servicio cumple con estándares legales de protección de información y si existe la opción de **despliegue on-premise**. Los LLMs en la nube suelen garantizar cumplimiento básico de GDPR, pero limitan el control directo. En cambio, los modelos auto-hospedados ofrecen **mayor soberanía sobre los datos**, aunque requieren más infraestructura y experiencia técnica. En un contexto universitario, suele ser más relevante encontrar un equilibrio entre **facilidad de uso, coste cero o reducido y garantías mínimas de privacidad**.

- Gemini y GPT ofrecen despliegues en la nube con cumplimiento GDPR, aunque el control on-premise es limitado.
- Los modelos de Hugging Face pueden auto-hospedarse, lo que da mayor control, pero a costa de infraestructura.
- En un contexto académico, la facilidad de uso y el plan gratuito de Gemini pesan más que el control absoluto.

Decisión final

La API seleccionada es **Gemini**, porque:

- Dispone de un plan gratuito generoso en número de llamadas.
- Ofrece calidad comparable a GPT-4 en muchos benchmarks, especialmente en español y razonamiento.
- Tiene SDKs oficiales para Python y JavaScript, encajando con el stack técnico del proyecto.
- Permite asegurar continuidad futura gracias a posibles acuerdos institucionales con la Universidad de Sevilla.
- Representa la opción más equilibrada entre calidad, coste, facilidad de integración y proyección académica.

Tabla de comparación detallada de APIs de LLM

Característica / API	Gemini	OpenAI GPT	DeepSeek	Hugging Face API	Cohere	Claude
Coste y accesibilidad	***	*	**	**	*	*
Calidad y rendimiento	***	***	**	**	**	***
Integración y soporte técnico	***	***	*	**	**	**
Privacidad y cumplimiento (RGPD)	**	**	*	***	**	**
Resultado global	* 11	* 9	* 6	* 9	* 7	* 8

Cuadro 2.3: Comparación de APIs de LLM

2.2– Infraestructura del sistema

La infraestructura técnica del sistema se basa en tecnologías de código abierto y servicios gratuitos que permiten una implementación funcional sin coste. Durante el desarrollo y pruebas del sistema, todos los componentes se ejecutan en entorno local, aunque se contempla la posibilidad de desplegarlos posteriormente en un entorno en la nube.

El framework Rasa se ejecuta en un servidor local utilizando Python como lenguaje base. Las acciones personalizadas del chatbot también están desarrolladas en Python y permiten la integración con otros servicios. Para la comprensión de lenguaje natural, se utiliza la API de Gemini, accedida mediante una clave gratuita proporcionada por Google AI Studio. El almacenamiento y recuperación de datos estructurados y vectoriales se realiza en Supabase, una plataforma que ofrece una base de datos PostgreSQL con soporte para la extensión pgvector. Esto permite centralizar tanto la información relacional (como horarios y datos de profesores) como los vectores semánticos necesarios para las búsquedas por similitud.

La interfaz web del asistente está desarrollada en React y se conecta al backend de Rasa mediante HTTP, funcionando como punto de interacción para el usuario final.

2.2.1. Arquitectura del sistema

La arquitectura del sistema propuesto se basa en un enfoque modular que combina un marco conversacional, técnicas de procesamiento del lenguaje natural mediante modelos generativos, y el uso de bases de datos tanto estructuradas como vectoriales. El sistema está diseñado para ofrecer respuestas útiles a los estudiantes de la Universidad de Sevilla a través de un asistente conversacional accesible desde una interfaz web.

Cuando un usuario realiza una consulta, esta es enviada al motor principal del sistema, construido sobre el framework Rasa, que se encarga de controlar el flujo de la conversación. Para la comprensión de la intención del usuario, el sistema puede operar en dos modos: uno basado en el entrenamiento tradicional de Rasa NLU con ejemplos, y otro utilizando la API del modelo de lenguaje Gemini para detectar la intención de forma dinámica.

Según la intención detectada, el sistema puede acceder a una base de datos estructurada (por ejemplo, para obtener información sobre horarios, profesorado, o fórmulas de calificación) o bien a una base de datos vectorial que contiene fragmentos semánticos extraídos de sitios web oficiales y documentación académica. Esta última permite realizar búsquedas por similitud y obtener información no estructurada de forma precisa.

Si el sistema no logra proporcionar una respuesta satisfactoria, se activa un mecanismo de recuperación (fallback) que consulta una base de datos de enlaces útiles asociados a cada tema. De este modo, se garantiza que el usuario siempre reciba algún tipo de orientación.

Tecnología	Rol en el sistema
Rasa	Framework principal para la gestión del diálogo y flujo de conversación.
Python	Lenguaje principal para el backend y acciones personalizadas.
Gemini (API)	Modelo generativo utilizado para la detección de intención y generación de respuestas naturales.
Supabase	Plataforma de base de datos que permite almacenar información estructurada y vectores semánticos.
pgvector	Extensión de PostgreSQL para el almacenamiento y búsqueda de vectores.
React	Desarrollo de la interfaz web del chatbot.
FAISS (opcional)	Motor local alternativo para búsqueda vectorial (solo si se opta por no usar Supabase).
Google AI Studio	Plataforma utilizada para obtener la API key de Gemini y gestionar el modelo.

Cuadro 2.4: Componentes tecnológicos del sistema

2.2.2. Componentes tecnológicos del sistema

2.3– Casos de uso principales

2.3.1. Consulta sobre horarios de clases

Consultar horario por asignatura

- Escenario positivo: El usuario introduce “*Horario de Bases de Datos II*” y el asistente devuelve su planificación semanal.
- Escenario negativo: El usuario consulta “*Horario de Bases de Datos 7*” (asignatura inexistente) y el asistente informa que no encuentra coincidencias.

Consultar horario por grupo

- Escenario positivo: El usuario pide “*Horario del Grupo A de Ingeniería de Software*” y obtiene la tabla completa.
- Escenario negativo: El usuario introduce un grupo no registrado (“*Grupo Z*”) y el asistente responde que ese grupo no existe.

Consultar horario por carrera/curso

- Escenario positivo: El usuario solicita “*Horario de 3º de Informática*” y recibe la planificación de todas las asignaturas de ese curso.
- Escenario negativo: El usuario introduce “*Horario de 9º de Informática*” y el asistente aclara que no existe tal curso.

2.3.2. Consulta sobre documentación necesaria para trámites

Consultar documentación para matrícula

- Escenario positivo: El usuario pregunta “¿*Qué necesito para matricularme en primero de carrera?*” y el asistente lista DNI, resguardo de pago, etc.
- Escenario negativo: El usuario introduce “*Necesito pasaporte para matrícula de nacional español*” y el asistente aclara que ese requisito no aplica.

Consultar documentación para becas

- Escenario positivo: El usuario pregunta “¿*Qué documentos hacen falta para la beca general?*” y el asistente devuelve la lista oficial (DNI, renta familiar, matrícula).
- Escenario negativo: El usuario consulta “*Puedo solicitar beca solo con una carta de recomendación*” y el asistente indica que no es suficiente.

Consultar documentación para certificados académicos

- Escenario positivo: El usuario pide “*Documentación necesaria para solicitar certificado de notas*” y el asistente responde con los requisitos (DNI, impreso de solicitud, pago de tasas).
- Escenario negativo: El usuario pregunta “*Puedo pedir certificado de notas sin identificarme*” y el asistente informa que no es posible.

Consultar documentación para convalidaciones/traslados

- Escenario positivo: El usuario solicita “*Requisitos para traslado de expediente*” y el asistente muestra la normativa aplicable.
- Escenario negativo: El usuario introduce “*Convalidar asignaturas sin entregar plan de estudios*” y el asistente explica que falta documentación esencial.

Comparar trámites similares (opcional)

- Escenario positivo: El usuario consulta “*Diferencia entre certificado simple y compulsado*” y el asistente explica la distinción.
- Escenario negativo: El usuario pide “*Compulsar sin documento original*” y el asistente indica que no se puede realizar.

2.3.3. Consulta sobre datos de contacto y tutorías del profesorado**Consultar despacho de un profesor**

- Escenario positivo: El usuario pregunta “*Despacho del profesor Juan Pérez*” y el asistente devuelve la ubicación.
- Escenario negativo: El usuario introduce un nombre inexistente y el asistente indica que no lo encuentra.

Consultar correo electrónico de un profesor

- Escenario positivo: El usuario pide “*Correo de la profesora Ana López*” y el asistente devuelve su email institucional.
- Escenario negativo: El usuario solicita “*Correo personal de la profesora Ana López*” y el asistente aclara que solo puede dar la dirección oficial.

Consultar horario de tutorías

- Escenario positivo: El usuario pregunta “*Horario de tutorías de Miguel García*” y el asistente devuelve día y hora.
- Escenario negativo: El usuario consulta fuera de rango (“*Tutorías a medianoche*”) y el asistente indica que no existen en ese horario.

Búsqueda por asignatura

- Escenario positivo: El usuario pregunta “¿Quién da Programación Avanzada?” y el asistente devuelve los profesores con sus datos de contacto.
- Escenario negativo: El usuario pide “Profesor de Programación Interplanetaria” (asignatura inexistente) y el asistente informa del error.

Búsqueda por departamento (opcional)

- Escenario positivo: El usuario solicita “Profesores del Departamento de Lenguajes y Sistemas Informáticos” y obtiene la lista.
- Escenario negativo: El usuario pide un departamento inexistente y el asistente indica que no figura en la base de datos.

2.3.4. Consulta sobre información de una asignatura**Consultar fórmula de calificación**

- Escenario positivo: El usuario pregunta “¿Cómo se califica Ingeniería del Software II?” y el asistente responde “40 % prácticas, 60 % examen”.
- Escenario negativo: El usuario introduce “100 % aprobado automático” y el asistente aclara que no existe esa modalidad.

Consultar sistema de evaluación

- Escenario positivo: El usuario pide “Sistema de evaluación de Matemáticas Discretas” y el asistente devuelve convocatorias y requisitos.
- Escenario negativo: El usuario consulta “Aprobar sin presentarse” y el asistente aclara que no es válido.

Consultar metodologías docentes

- Escenario positivo: El usuario pregunta “Metodología de Programación Avanzada” y el asistente devuelve que es aprendizaje basado en proyectos.
- Escenario negativo: El usuario pide “Metodología libre, sin clases” y el asistente indica que no corresponde con la asignatura.

Consultar bibliografía recomendada

- Escenario positivo: El usuario pide “Bibliografía de Sistemas Operativos” y el asistente devuelve libros y artículos recomendados.
- Escenario negativo: El usuario solicita “Bibliografía de TikTok” y el asistente aclara que no es material académico válido.

Consultar competencias de la asignatura (opcional)

- Escenario positivo: El usuario pregunta “Competencias de Bases de Datos I” y el asistente responde con los objetivos formativos.
- Escenario negativo: El usuario pide “Competencias de Asignatura XYZ inventada” y el asistente aclara que no existe.

2.3.5. Consulta sobre normas y documentación de la Universidad

Consultar convocatorias de examen

- Escenario positivo: El usuario pregunta “*¿Cuántas convocatorias de examen hay en cada asignatura de grado?*” y el asistente responde que, por norma, existen dos por curso.
- Escenario negativo: El usuario pide “*Convocatorias ilimitadas*” y el asistente aclara que no es correcto.

Consultar tiempos de examen

- Escenario positivo: El usuario consulta “*Duración del examen oficial de Matemáticas I*” y el asistente responde que son 2 horas.
- Escenario negativo: El usuario pregunta “*Examen de 10 minutos*” y el asistente indica que ese dato no es válido.

Consultar número máximo de suspensos permitidos

- Escenario positivo: El usuario pide “*¿Cuántas veces se puede suspender una asignatura antes de perder el derecho a examen?*” y el asistente responde que hay un límite (ej. 6).
- Escenario negativo: El usuario consulta “*Suspender infinitas veces*” y el asistente informa que no es posible.

Consultar plazos de revisión de exámenes (opcional)

- Escenario positivo: El usuario pregunta “*¿Cuántos días tengo para pedir revisión de examen tras publicarse las notas?*” y el asistente responde que son 7 días hábiles.
- Escenario negativo: El usuario consulta “*Revisar un examen de hace 3 años*” y el asistente aclara que el plazo ya expiró.

Conceptos iniciales

- 3.1– Sistema conversacional y componentes de Rasa**
- 3.2– Procesamiento del lenguaje natural**
- 3.3– Gestión del diálogo**
- 3.4– Backend y persistencia de datos**

Parte II

Metodología y Planificación

CAPÍTULO 4

Metodología

4.1– SCRUM ajustado en la ejecución

4.2– Herramientas

Planificación

Este capítulo presenta la planificación del proyecto Linceus Assistant. La organización del trabajo sigue el ciclo de vida clásico de un proyecto de ingeniería del software, dividido en **tres fases**: *Inicio*, *Desarrollo* y *Cierre*. Esta estructura es coherente con la guía PMBOK y con la práctica habitual en proyectos académicos, y permite separar el trabajo de planificación previa, el desarrollo iterativo del producto y la validación con usuarios y entrega final.

Dentro de cada fase se identifican **paquetes de trabajo** que agrupan tareas con un objetivo común y un entregable concreto. Las épicas de desarrollo en sentido SCRUM (asignaturas, profesores, horarios, etc.) se ubican en la fase de Desarrollo y se ejecutan de forma incremental sobre sprints de 3–4 semanas, según se detalla en el Capítulo 4.

5.1– Estructura de Descomposición del Trabajo (EDT)

La Estructura de Descomposición del Trabajo (EDT), o *Work Breakdown Structure* (WBS), organiza jerárquicamente todo el trabajo necesario para completar el proyecto. El nivel 0 representa el proyecto completo, el nivel 1 las tres fases del ciclo de vida y el nivel 2 los paquetes de trabajo dentro de cada fase. Las tareas individuales (nivel 3) se detallan en el diccionario EDT de la Sección 5.2.

La Figura 5.1 muestra la EDT a nivel de fases y paquetes de trabajo.

5.2– Diccionario EDT

El diccionario EDT define cada paquete de trabajo y sus tareas constituyentes, especificando descripción, entregables y estimación de esfuerzo. Las estimaciones siguen la escala: **S** (Small, ≤ 1 semana), **M** (Medium, 1–2 semanas) y **L** (Large, 2–3 semanas).

5.2.1. Fase 1 — Inicio

La fase de inicio agrupa todo el trabajo previo a la implementación: planificación, estudio del estado del arte, análisis de tecnologías y diseño arquitectónico. Es la fase más corta en duración pero condiciona todas las decisiones posteriores.

1.1 — Gestión del proyecto

Paquete transversal: aunque su esfuerzo principal se concentra en el inicio (planificación) y en el cierre (memoria, defensa), la gestión del proyecto está activa durante todo el ciclo de vida.



Figura 5.1: Estructura de Descomposición del Trabajo (EDT) — fases y paquetes de trabajo.

ID	Tarea	Descripción y entregables	Est.
1.1.1	Anteproyecto y planificación	Definición del alcance, objetivos, EDT inicial y cronograma orientativo. <i>Entregable: anteproyecto aprobado.</i>	M
1.1.2	Seguimiento iterativo	Revisión del progreso al cierre de cada sprint, ajuste del backlog y gestión de riesgos. <i>Entregable: registro de decisiones (docs/sprints/).</i>	L
1.1.3	Comunicación con tutor	Reuniones periódicas de seguimiento, actas y validación de hitos. <i>Entregable: actas de seguimiento.</i>	M

Cuadro 5.1: Diccionario EDT — 1.1: Gestión del proyecto.

1.2 — Investigación tecnológica

Análisis del estado del arte y comparativa de tecnologías, base de las decisiones de diseño documentadas en el Capítulo 6.

ID	Tarea	Descripción y entregables	Est.
1.2.1	Estado del arte de chatbots	Revisión de chatbots universitarios publicados (AdmitBot, EDUBOT/LiSA, Ranoliya FAQ, Dibya & Sahoo) y umbrales académicos de calidad. <i>Entregable: marco bibliográfico.</i>	M
1.2.2	Comparativa de frameworks	Análisis de Rasa, Botpress y DeepPavlov según coste, integración Python, RAG y despliegue local. <i>Entregable: tabla comparativa, decisión documentada.</i>	L
1.2.3	Comparativa de LLM/embeddings	Análisis de Gemini, OpenAI, DeepSeek y Ollama según coste, calidad y privacidad. <i>Entregable: tabla comparativa.</i>	M
1.2.4	Setup del entorno	Instalación de Python, Rasa, Supabase, Gemini API, Git/GitHub. Estructura inicial del repositorio. <i>Entregable: entorno funcional reproducible.</i>	S

Cuadro 5.2: Diccionario EDT — 1.2: Investigación tecnológica.

1.3 — Diseño de la arquitectura

Definición de la arquitectura por capas, modelo de datos y contratos entre componentes antes de empezar el desarrollo.

ID	Tarea	Descripción y entregables	Est.
1.3.1	Arquitectura por capas	Definición de las capas (presentación, NLU, lógica de negocio, IA, datos, admin) y de las interfaces entre ellas. <i>Entregable: diagrama de componentes.</i>	M
1.3.2	Diagrama de despliegue	Modelo de despliegue sobre DigitalOcean con contenedores Docker, Supabase como BD gestionada y Gemini como servicio externo. <i>Entregable: diagrama de despliegue.</i>	S
1.3.3	Modelo de datos	Diseño del esquema relacional (asignaturas, profesores, horarios, planes docentes) y vectorial (chunks + embeddings con pgvector). <i>Entregable: diagrama ER.</i>	M

Cuadro 5.3: Diccionario EDT — 1.3: Diseño de la arquitectura.

5.2.2. Fase 2 — Desarrollo

Fase central del proyecto. Se ejecuta de forma iterativa siguiendo el marco SCRUM adaptado descrito en el Capítulo 4, con sprints de 3–4 semanas. Cada paquete corresponde a una épica funcional del producto y se cierra con un incremento entregable.

2.1 — Épica Asignaturas

Funcionalidad nuclear: consultas sobre asignaturas (ficha, listado, conteo) mediante *Text-to-SQL* sobre la base de datos relacional y RAG sobre los planes docentes.

2.2 — Épica Profesores

Búsqueda de profesorado por nombre o por asignatura con datos de contacto, despacho, departamento y atajo de coordinador/suplente vía RAG sobre el plan docente.

ID	Tarea	Descripción y entregables	Est.
2.1.1	Esquema y carga de datos	Tablas asignaturas, titulaciones, centros y carga inicial de las tres titulaciones GII de la ETSII. <i>Entregable: scripts SQL + datos.</i>	M
2.1.2	NLU de asignaturas	Intents (consulta_asignatura_especifica, consulta_asignaturas_listado, consulta_asignaturas_conteo) y entidades. <i>Entregable: ficheros NLU.</i>	M
2.1.3	Acción Text-to-SQL	Generación dinámica de SQL con Gemini, validación, ejecución y formateo de la respuesta natural. <i>Entregable: módulo actions/asignaturas/.</i>	L
2.1.4	Resolución de seguimiento	Herencia de slot ultimo_nombre_asignatura para preguntas elípticas (“y cuántos créditos?”). <i>Entregable: resolver_asignatura.</i>	M
2.1.5	Testing de la épica	Suite de aceptación con ≥ 30 casos golden + casos negativos y con typos. <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.4: Diccionario EDT — 2.1: Épica Asignaturas.

ID	Tarea	Descripción y entregables	Est.
2.2.1	Esquema profesores	Tablas profesores, departamentos, profesor_asignatura con FK hacia asignaturas. <i>Entregable: script SQL.</i>	S
2.2.2	Importación desde us.es PDI	Scraper del directorio oficial de la US para importar profesorado de la ETSII; deduplicación por email. <i>Entregable: scraper integrado en panel admin.</i>	L
2.2.3	NLU y acción	Intent consulta_profesor con pipeline Text-to-SQL + fallback RAG sobre plan docente. <i>Entregable: actions/profesores/.</i>	L
2.2.4	Testing de la épica	Suite con 43 casos cubriendo nombre, asignatura, tutorías y coordinador. <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.5: Diccionario EDT — 2.2: Épica Profesores.

2.3 — Épica Horarios

Consultas de horario por curso/grupo o por asignatura, con extracción automática del PDF oficial de horarios de la ETSII.

ID	Tarea	Descripción y entregables	Est.
2.3.1	Esquema horarios	Tablas <code>grupos.clase</code> , <code>horarios</code> , <code>aulas</code> con cuatrimestre como dimensión. <i>Entregable: script SQL.</i>	M
2.3.2	Extractor PDF ETSII	Parser con <code>pdfplumber</code> de la cuadrícula oficial; reconoce códigos <code>xGy-Cz</code> y separa teoría de laboratorio. <i>Entregable: admin/horarios_extractores/.</i>	L
2.3.3	NLU y acciones	Intents <code>consulta_horario (curso+grupo)</code> y <code>consulta_horario_asignatura</code> ; herencia de slot para seguimiento. <i>Entregable: actions/horarios/.</i>	L
2.3.4	Testing de la épica	Suite con 45 casos (horario semanal, por día, con/sin cuatrimestre, con typos). <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.6: Diccionario EDT — 2.3: Épica Horarios.

2.4 — Pipeline RAG

Infraestructura transversal de *Retrieval Augmented Generation* sobre los planes docentes de las asignaturas, utilizada por las épicas de asignaturas y profesores.

ID	Tarea	Descripción y entregables	Est.
2.4.1	Extracción y <i>chunking</i>	Parseo de planes docentes (HTML/PDF), <i>chunking</i> semántico con metadatos (asignatura, sección, año). <i>Entregable: pipeline.</i>	M
2.4.2	Embeddings y <code>pgvector</code>	Generación con <code>text-embedding-004</code> y almacenamiento en <code>pgvector</code> con índices HNSW; función SQL <code>buscar_plan_docente()</code> . <i>Entregable: pipeline de vectorización.</i>	M
2.4.3	Action RAG	Acción que combina retrieval semántico con re-ranking por sección y umbral de similitud; respuesta con citas. <i>Entregable: módulo RAG en actions/.</i>	L
2.4.4	Anti-alucinación	Prompts reforzados que prohíben inventar nombres, emails o aulas no presentes en los chunks. <i>Entregable: prompts revisados.</i>	M

Cuadro 5.7: Diccionario EDT — 2.4: Pipeline RAG.

2.5 — Panel admin y frontend

Interfaces visibles del producto: el widget de chat para el alumno y el panel de administración para mantener los datos curados.

5.2.3. Fase 3 — Cierre

Fase final del proyecto: validación funcional exhaustiva, despliegue en producción, ejecución del piloto con usuarios reales y entrega de la documentación académica.

ID	Tarea	Descripción y entregables	Est.
2.5.1	Widget de chat	SPA en JavaScript vanilla, embebible mediante <code><script></code> , con renderizado Markdown y botones rápidos. <i>Entregable: frontend/</i> .	M
2.5.2	API admin (Flask)	Endpoints REST para CRUD de centros, titulaciones, asignaturas, profesores y planes docentes. <i>Entregable: admin/</i> .	L
2.5.3	SPA admin	Interfaz web con navegación jerárquica Centro → Titulación → Asignatura. <i>Entregable: frontend/admin.*</i> .	M
2.5.4	Integración de scrapers	Scrapers de Sevius y us.es expuestos en el panel con flujo <i>preview → insert</i> . <i>Entregable: rutas /sync</i> .	M

Cuadro 5.8: Diccionario EDT — 2.5: Panel admin y frontend.

3.1 — Validación y testing

Suite de pruebas multinivel siguiendo el modelo de calidad ISO/IEC 25010 y la metodología *Behavior-Driven Development*.

ID	Tarea	Descripción y entregables	Est.
3.1.1	Plan de aceptación	Diseño del plan de pruebas según ISO 25010 + BDD: positivos, negativos, edge, robustez. <i>Entregable: plans/aceptacion-prototipo.md</i> .	M
3.1.2	Suite E2E	159 casos sobre el stack completo (NLU + actions + SQL + RAG + LLM). <i>Entregable: 94,3 % PASS</i> .	L
3.1.3	RAG retrieval + profundidad	Capa baseline (50 preguntas) y profundidad (74 preguntas reales sobre 19 asignaturas). <i>Entregables: 94,0 % y 91,9 %</i> .	M
3.1.4	NLU + Policy	Validación con <i>rasa test core</i> sobre 44 stories. <i>Entregable: 100 % accuracy de acciones</i> .	S
3.1.5	Métricas no funcionales	Latencia p50/p95 y coste por turno con extrapolación a 700 alumnos. <i>Entregable: coste_latencia.md</i> .	M

Cuadro 5.9: Diccionario EDT — 3.1: Validación y testing.

3.2 — Piloto con usuarios

Despliegue del prototipo y validación con conversaciones reales de alumnos de la ETSII.

ID	Tarea	Descripción y entregables	Est.
3.2.1	Captura de tráfico real	Registro de las conversaciones piloto en <i>conversation_log</i> . <i>Entregable: 59 sesiones / 203 turnos</i> .	S
3.2.2	Auditoría manual	Replay turno a turno desde el panel admin; marcado de sesiones validadas. <i>Entregable: 58/59 sesiones validadas (98,3 %)</i> .	M
3.2.3	Iteración correctiva	Fixes detectados durante la auditoría (alias compactado, anti-alucinación, fecha actual en prompt, seguimiento cross-intent). <i>Entregable: registro de decisiones D-064..D-069</i> .	M

Cuadro 5.10: Diccionario EDT — 3.2: Piloto con usuarios.

3.3 — Despliegue en producción

Puesta en funcionamiento del sistema en una infraestructura accesible públicamente y configuración del pipeline de despliegue continuo.

ID	Tarea	Descripción y entregables	Est.
3.3.1	Provisión del droplet	Aprovisionamiento del servidor en DigitalOcean (Ubuntu 24.04 LTS), <i>firewall</i> y dominio. <i>Entregable: servidor accesible.</i>	S
3.3.2	Containerización	<code>docker-compose</code> con cuatro servicios (nginx, rasa, actions, admin) e imagen Rasa custom con spaCy. <i>Entregable: docker/.</i>	M
3.3.3	Pipeline CI/CD	<i>Workflow</i> de GitHub Actions que despliega vía SSH al droplet en cada <i>merge</i> a main. <i>Entregable: deploy.yml.</i>	S

Cuadro 5.11: Diccionario EDT — 3.3: Despliegue en producción.

3.4 — Memoria y defensa

Documentación académica final del proyecto y preparación del acto de defensa.

ID	Tarea	Descripción y entregables	Est.
3.4.1	Redacción de la memoria	Capítulos de introducción, marco, metodología, arquitectura, planificación, pruebas y conclusiones. <i>Entregable: memoria.</i>	L
3.4.2	Diagramas técnicos	Diagrama de componentes y de despliegue en notación UML2 (PlantUML). <i>Entregable: img/diagrama-*.png.</i>	S
3.4.3	Defensa	Diapositivas, ensayo y demo en vivo del prototipo desplegado. <i>Entregable: presentación.</i>	M

Cuadro 5.12: Diccionario EDT — 3.4: Memoria y defensa.

5.3– Resumen cuantitativo de la EDT

La Tabla 5.13 resume los paquetes de trabajo por fase y la distribución del esfuerzo estimado.

5.4– Backlog del producto

El backlog del producto se deriva de los paquetes de la fase de Desarrollo y los organiza según su prioridad de implementación. Cada épica se desarrolla de forma incremental siguiendo el marco SCRUM adaptado descrito en el Capítulo 4, de modo que las funcionalidades se entregan en *releases* sucesivas validables por separado.

La priorización responde a dos criterios:

1. **Valor para el usuario:** Las épicas que cubren las consultas más frecuentes de los estudiantes (asignaturas, profesorado, horarios) se implementan primero.
2. **Dependencia técnica:** La infraestructura transversal (pipeline RAG, panel admin) se desarrolla en paralelo a las épicas que la consumen, pero su *release* depende de tener al menos una épica funcional sobre la que probarla.

La Tabla 5.14 muestra el orden de prioridad de las épicas dentro de la fase de Desarrollo.

Fase	Paquete de trabajo	Tareas	S	M	L
1. Inicio	1.1 Gestión del proyecto	3	0	2	1
	1.2 Investigación tecnológica	4	1	2	1
	1.3 Diseño de la arquitectura	3	1	2	0
2. Desarrollo	2.1 Épica Asignaturas	5	0	4	1
	2.2 Épica Profesores	4	1	1	2
	2.3 Épica Horarios	4	0	2	2
	2.4 Pipeline RAG	4	0	3	1
	2.5 Panel admin y frontend	4	0	3	1
3. Cierre	3.1 Validación y testing	5	1	3	1
	3.2 Piloto con usuarios	3	1	2	0
	3.3 Despliegue en producción	3	2	1	0
	3.4 Memoria y defensa	3	1	1	1
Total		45	8	26	11

Cuadro 5.13: Resumen cuantitativo de la EDT por fase y paquete de trabajo.

Prioridad	Épica / paquete	Dependencia
1	2.1 Asignaturas (BD relacional + Text-to-SQL)	—
2	2.2 Profesores (BD + RAG plan docente)	2.1 (esquema BD compartido)
3	2.4 Pipeline RAG (transversal)	2.1 (inventario de fuentes)
4	2.3 Horarios (BD + extractor PDF)	2.1 (esquema BD)
5	2.5 Panel admin y frontend	Todas las anteriores

Cuadro 5.14: Priorización del backlog dentro de la fase de Desarrollo.

5.5– Planificación de sprints

El proyecto se organiza en sprints de **3–4 semanas de duración**, siguiendo una adaptación de SCRUM para un proyecto individual (véase Capítulo 4). Cada sprint cierra con un incremento funcional del sistema y un breve registro de decisiones técnicas.

5.6– Cronograma

Parte III

Diseño e Implementación

Arquitectura del sistema y decisiones de diseño

Este capítulo describe la arquitectura del sistema LinceUS Assistant, las decisiones de diseño adoptadas durante su desarrollo y los diagramas técnicos que representan la estructura de componentes y el modelo de despliegue. El objetivo es ofrecer una visión completa de cómo se organiza internamente el sistema, cómo se comunican sus partes y qué criterios han guiado las principales elecciones técnicas.

6.1– Arquitectura general

LinceUS Assistant sigue una **arquitectura por capas** (*layered architecture*) con tres niveles claramente diferenciados: capa de presentación, capa de procesamiento conversacional y capa de datos. Esta separación permite que cada capa evolucione de forma independiente y que los cambios en una no afecten directamente a las demás, siempre que se respeten las interfaces definidas entre ellas.

6.1.1. Capa de presentación

La capa de presentación es la interfaz visible para el usuario final. Consiste en un **widget de chat web** desarrollado en JavaScript puro (vanilla JS), HTML y CSS, diseñado como un componente autocontenido y embebible en cualquier página institucional de la Universidad de Sevilla.

Las principales características de esta capa son:

- **Independencia tecnológica:** El widget no depende de frameworks como React o Angular; se implementa con JavaScript estándar (ES6+), lo que minimiza las dependencias y facilita su integración en páginas web existentes mediante una única etiqueta `<script>`.
- **Comunicación REST:** El widget se comunica con el servidor Rasa mediante peticiones HTTP POST al endpoint `/webhooks/rest/webhook`, enviando mensajes del usuario y recibiendo las respuestas del asistente en formato JSON.
- **Renderizado enriquecido:** Soporta formato Markdown en las respuestas del bot, incluyendo tablas, enlaces y listas, para presentar la información académica de forma clara y estructurada.
- **Diseño institucional:** Emplea la identidad visual de la Universidad de Sevilla (colores corporativos, logotipo) para integrarse visualmente en el entorno universitario.

6.1.2. Capa de procesamiento conversacional

Esta capa constituye el núcleo del sistema y está compuesta por varios servicios que colaboran entre sí:

- **Rasa Server:** Motor principal de comprensión del lenguaje natural (NLU) y gestión de diálogo (Core). Recibe los mensajes del usuario, los procesa a través del pipeline NLU para extraer intenciones y entidades, y decide la siguiente acción a ejecutar mediante las políticas de diálogo configuradas.
- **Action Server (Rasa SDK):** Servidor independiente que ejecuta las acciones personalizadas escritas en Python. Cuando Rasa Core determina que debe ejecutarse una acción personalizada (por ejemplo, consultar la base de datos), delega la ejecución en este servidor a través de una petición HTTP al endpoint `/webhook`.
- **Google Gemini API:** Servicio externo utilizado para dos propósitos: (1) generación de consultas SQL a partir de lenguaje natural (*Text-to-SQL*) mediante el modelo `gemma-3-27b-it`, y (2) generación de embeddings vectoriales mediante el modelo `gemini-embedding-001` para la búsqueda semántica en el pipeline RAG.
- **Ollama (opcional):** Servicio local de inferencia de modelos LLM que actúa como alternativa a Gemini cuando la API externa no está disponible o se agotan las cuotas de uso. Permite ejecutar modelos como `llama3.2:3b` en CPU sin depender de servicios en la nube.

6.1.3. Capa de datos

La capa de datos se implementa sobre **Supabase**, una plataforma que proporciona una instancia gestionada de PostgreSQL con extensiones adicionales:

- **PostgreSQL:** Almacena toda la información académica estructurada: universidades, centros, titulaciones, asignaturas, planes docentes y sus metadatos asociados.
- **pgvector:** Extensión que añade el tipo de dato `vector` y operadores de similitud (distancia coseno, euclidiana y producto punto). Almacena los embeddings de los fragmentos de documentos vectorizados y permite realizar búsquedas semánticas eficientes mediante índices HNSW.
- **Funciones SQL personalizadas:** Funciones almacenadas en la base de datos que encapsulan la lógica de búsqueda vectorial (`buscar_plan_docente()`, `buscar_plan_docente_global()`), permitiendo filtrar por asignatura, grupo y curso académico con un umbral de similitud configurable.

6.2– Diagrama de componentes

El diagrama de componentes muestra los módulos principales del sistema, sus responsabilidades y las interfaces a través de las cuales se comunican. La Figura 6.1 representa esta vista.

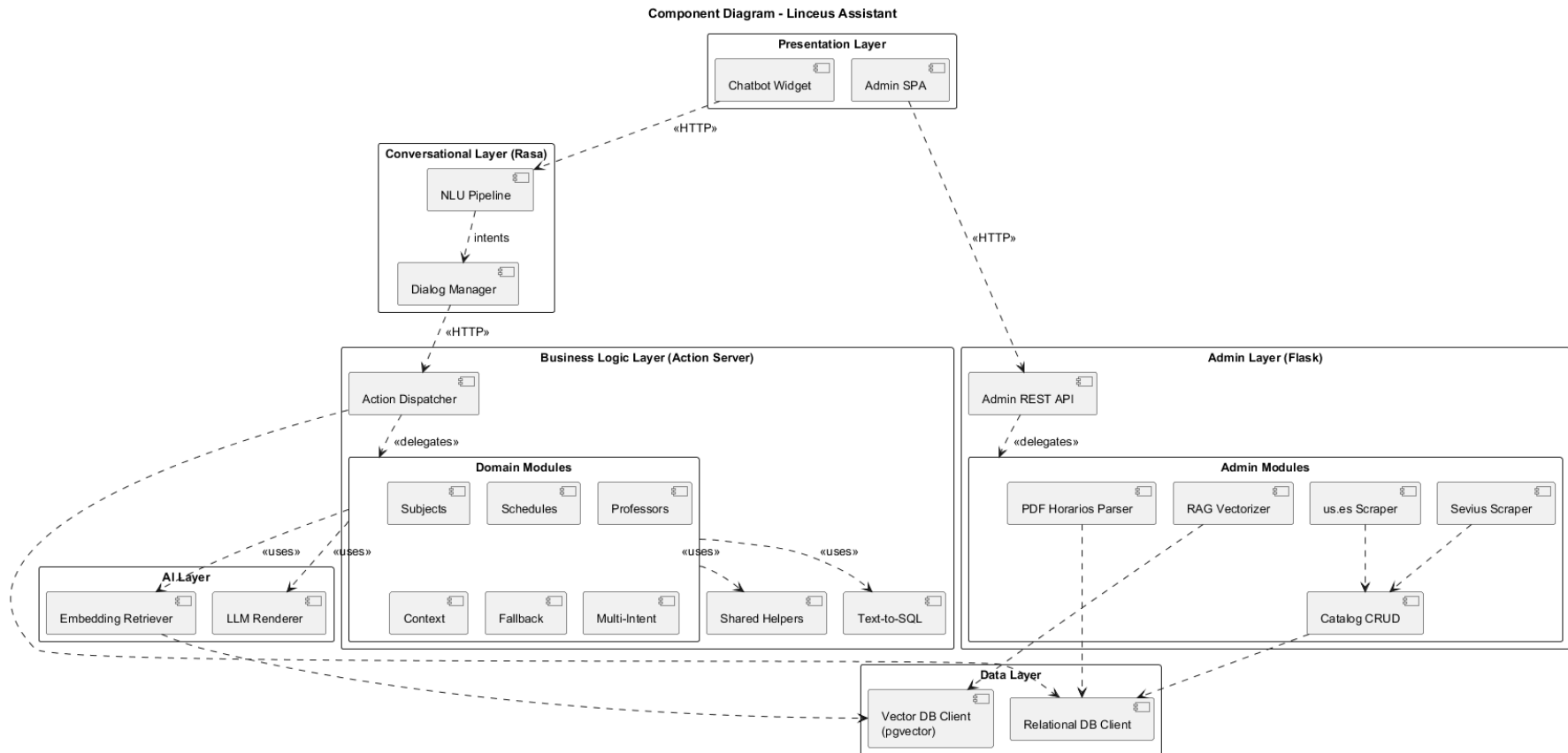


Figura 6.1: Diagrama de componentes del sistema Linceus.

A continuación se describe la responsabilidad de cada componente y las interfaces entre ellos:

- **Chat Widget** → **Rasa NLU**: El widget envía el texto del usuario mediante una petición HTTP POST al endpoint REST de Rasa. La comunicación es síncrona: el widget espera la respuesta para mostrarla en la interfaz.
- **Rasa NLU** → **Rasa Core**: Tras procesar el mensaje a través del pipeline NLU (tokenización, extracción de características, clasificación de intención y extracción de entidades), el resultado se pasa al módulo Core, que decide la siguiente acción.
- **Rasa Core** → **Action Server**: Cuando la acción seleccionada es personalizada (prefijo `action_`), Core envía una petición HTTP al Action Server con el tracker completo de la conversación (historial, slots, última intención y entidades).
- **Action Server** → **Text-to-SQL**: Para consultas sobre asignaturas, el Action Server invoca el motor Text-to-SQL, que construye un prompt con el esquema de la base de datos y la pregunta del usuario, lo envía a Gemini y recibe una consulta SQL validada.
- **Action Server** → **Gemini / Ollama**: El Action Server puede invocar Gemini para generación de SQL, detección de titulación o generación de embeddings. Si Gemini no está disponible, se recurre a Ollama como alternativa local.
- **RAG Pipeline** → **pgvector**: La búsqueda semántica genera un embedding de la pregunta del usuario y lo compara contra los embeddings almacenados en pgvector usando similitud coseno, con un umbral mínimo de 0.5.
- **Text-to-SQL** → **PostgreSQL**: Las consultas SQL generadas se ejecutan directamente contra la base de datos PostgreSQL de Supabase, con filtros de seguridad inyectados automáticamente (restricción por titulación).

6.3– Diagrama de despliegue

El diagrama de despliegue describe la distribución física de los componentes del sistema en nodos de ejecución y las conexiones de red entre ellos. La Figura 6.2 muestra esta configuración.

6.3.1. Nodos de ejecución

El sistema se despliega sobre los siguientes nodos:

1. **Navegador web del usuario**: Ejecuta el widget de chat (JavaScript) y se comunica con el servidor Rasa mediante HTTP. No requiere instalación alguna por parte del usuario.
2. **Máquina local de desarrollo**: Alberga los tres servicios principales del backend:
 - *Rasa Server* (`localhost:5005`): Sirve la API REST del chatbot y carga el modelo entrenado (`.tar.gz`).
 - *Action Server* (`localhost:5055`): Ejecuta las acciones personalizadas con un timeout de 300 segundos para acomodar consultas LLM en CPU.
 - *Ollama* (`localhost:11434`): Servicio opcional de inferencia local de modelos LLM.
 - *RAG Pipeline*: Proceso batch que se ejecuta bajo demanda para vectorizar nuevos documentos PDF.

3. **Supabase Cloud (AWS eu-west-3):** Instancia gestionada de PostgreSQL con pgvector, accesible mediante connection pooling en el puerto 6543. Aloja tanto los datos estructurados como los embeddings vectoriales.
4. **Google Cloud:** Proporciona acceso a la API de Gemini para generación de texto (Text-to-SQL) y generación de embeddings. La comunicación se realiza mediante HTTPS con autenticación por API key.

6.3.2. Protocolos de comunicación

Origen	Destino	Protocolo	Puerto
Navegador	Rasa Server	HTTP POST (REST)	5005
Rasa Server	Action Server	HTTP POST (webhook)	5055
Action Server	Supabase	TCP (psycopg2)	6543
Action Server	Gemini API	HTTPS (REST)	443
Action Server	Ollama	HTTP (REST)	11434
RAG Pipeline	Supabase	TCP (psycopg2)	6543
RAG Pipeline	Gemini Embeddings	HTTPS (REST)	443

Cuadro 6.1: Protocolos y puertos de comunicación entre nodos del sistema.

6.4– Decisiones de diseño

A lo largo del desarrollo del sistema se han tomado decisiones de diseño que han condicionado la arquitectura, el rendimiento y la mantenibilidad del proyecto. En esta sección se documentan las más relevantes, indicando el contexto, las alternativas consideradas y la justificación de cada elección.

6.4.1. Text-to-SQL con LLM en lugar de reglas estáticas

Contexto: El sistema debe responder consultas sobre asignaturas que pueden formularse de formas muy diversas (por nombre, curso, tipología, créditos, duración o combinaciones de estos filtros). Implementar todas las variantes mediante reglas *if-else* o expresiones regulares resultaría en un código excesivamente largo, frágil y difícil de mantener.

Decisión: Se optó por un enfoque **Text-to-SQL basado en LLM**, donde el modelo Gemini (gemma-3-27b-it) recibe un prompt que incluye el esquema de la tabla `asignaturas`, las columnas permitidas y la pregunta del usuario, y genera la consulta SQL correspondiente.

Justificación:

- **Flexibilidad:** El modelo interpreta preguntas en lenguaje natural sin necesidad de anticipar todas las formulaciones posibles.
- **Mantenibilidad:** Añadir nuevos atributos o filtros solo requiere actualizar el prompt, no reescribir lógica de parsing.
- **Seguridad:** Se aplican mecanismos de protección como una *whitelist* de columnas permitidas, prohibición de subconsultas e inyección automática del filtro de titulación.

Temperatura 0.0: Se configura el modelo con temperatura cero para obtener respuestas deterministas, crítico para la generación de SQL donde la variabilidad es indeseable.

6.4.2. Gemini como servicio principal con Ollama como alternativa local

Contexto: El sistema requiere un modelo de lenguaje grande para Text-to-SQL y generación de embeddings. Depender exclusivamente de una API externa implica riesgos de disponibilidad y costes, mientras que depender solo de ejecución local exige hardware potente.

Decisión: Se adoptó una **estrategia dual**: Gemini como servicio principal (plan gratuito con 15.000 peticiones/día) y Ollama como alternativa local para situaciones donde la API externa no esté disponible.

Justificación:

- El plan gratuito de Gemini es suficiente para el volumen de uso académico del proyecto.
- Ollama con `llama3.2:3b` permite seguir operando sin conexión a internet, aunque con menor calidad en las respuestas.
- Esta dualidad alinea el proyecto con los requisitos de privacidad (RGPD), ya que el flujo local no envía datos a terceros.

6.4.3. Embeddings de 2000 dimensiones con HNSW

Contexto: El modelo `gemini-embedding-001` genera embeddings de 3072 dimensiones de forma nativa. Sin embargo, el índice HNSW de `pgvector` presenta limitaciones de rendimiento y almacenamiento con dimensionalidades muy altas.

Decisión: Se redujo la dimensionalidad de salida a **2000 dimensiones** mediante el parámetro `output_dimensionality` de la API de Gemini.

Justificación:

- 2000 dimensiones ofrecen un equilibrio entre la calidad de la representación semántica y la eficiencia del índice HNSW (parámetros $m = 16$, $ef_construction = 64$).
- Reduce el tamaño de almacenamiento por vector en aproximadamente un 35 %.
- Las pruebas empíricas muestran que la pérdida de precisión en la búsqueda semántica es negligible para el corpus de documentos del proyecto (planes docentes de la ETSII).

6.4.4. Chunking con solapamiento para preservar contexto

Contexto: Los planes docentes en PDF contienen información densa que debe fragmentarse para su vectorización. Un chunking demasiado grande diluye la relevancia; uno demasiado pequeño pierde contexto.

Decisión: Se implementó un chunking de **800 caracteres con 100 caracteres de solapamiento** y un mínimo de 50 caracteres por fragmento.

Justificación:

- 800 caracteres ($\approx 150\text{--}200$ tokens) es suficiente para capturar un párrafo o sección temática completa.
- El solapamiento de 100 caracteres garantiza que la información en los límites de fragmentos no se pierda, preservando la coherencia semántica.
- Se aplica detección de secciones por expresiones regulares (“Datos básicos”, “Objetivos”, “Contenidos”, “Evaluación”, “Bibliografía”) para etiquetar cada fragmento con su sección de origen.

6.4.5. Búsqueda vectorial con fallback a búsqueda por palabras clave

Contexto: La generación de embeddings para las consultas de búsqueda depende de la API de Gemini, que puede agotar su cuota diaria (100 peticiones por minuto, 1000 por día).

Decisión: Se implementó un sistema de **dobles búsquedas**: búsqueda vectorial como método principal y búsqueda por palabras clave (ILIKE) como fallback automático.

Justificación:

- La búsqueda vectorial ofrece mejor precisión semántica (encuentra “sistema de evaluación” cuando el usuario pregunta “¿cómo se califica?”).
- El fallback por palabras clave garantiza que el sistema siga operativo aunque la API de embeddings no esté disponible, con resultados razonables para consultas que contienen términos exactos.
- El umbral de similitud coseno se fija en 0.5, bajo el cual los resultados se descartan por considerarse irrelevantes.

6.4.6. Fuzzy matching para resolución de nombres

Contexto: Los usuarios introducen nombres de asignaturas con errores ortográficos, sin tildes, con abreviaturas o con variaciones coloquiales (“FP” por “Fundamentos de Programación”, “ADDA” por “Análisis y Diseño de Datos y Algoritmos”).

Decisión: Se integró la librería **RapidFuzz** con un umbral de similitud del 80 % combinado con una tabla de alias y acrónimos mantenida en el código.

Justificación:

- El algoritmo de distancia de Levenshtein de RapidFuzz tolera errores tipográficos habituales sin requerir entrenamiento adicional.
- La tabla de alias cubre las abreviaturas más comunes utilizadas por los estudiantes de la ETSII.
- Si la coincidencia difusa falla, se recurre al LLM (Gemini) como último recurso para intentar identificar la asignatura.

6.4.7. Slots conversacionales para memoria de contexto

Contexto: Los usuarios formulan preguntas de seguimiento que hacen referencia implícita a asignaturas o titulaciones mencionadas previamente (“¿Y cuántos créditos tiene?”, “¿Es obligatoria?”).

Decisión: Se utilizan **slots de Rasa** para mantener el estado de la conversación:

- `contexto_titulacion`: Titulación activa seleccionada por el usuario.
- `ultimo_codigo_consultado`: Código de la última asignatura consultada.
- `ultimo_nombre_asignatura`: Nombre de la última asignatura mencionada.
- `ultimos_resultados_asignaturas`: Lista de resultados de la última consulta (para paginación).

Justificación:

- Los slots permiten que las acciones personalizadas recuperen el contexto sin que el usuario deba repetir información.

- La separación entre código y nombre de asignatura facilita tanto las consultas SQL (por código) como las respuestas al usuario (por nombre legible).
- La titulación por defecto es GII-IS (Ingeniería del Software), configurable por el usuario mediante la intención `cambiar_contexto_academico`.

6.4.8. Pipeline NLU de 9 etapas con doble featurización

Contexto: El pipeline NLU de Rasa debe ser capaz de clasificar correctamente intenciones en español, tolerando errores ortográficos, abreviaturas y variaciones léxicas propias del lenguaje coloquial estudiantil.

Decisión: Se configuró un pipeline de 9 componentes que combina **featurización a nivel de palabra** (n-gramas 1–2) y **a nivel de carácter** (n-gramas 1–4):

1. `WhitespaceTokenizer`: Tokenización por espacios en blanco.
2. `RegexFeaturizer`: Patrones regex para códigos de asignatura y formatos especiales.
3. `LexicalSyntacticFeaturizer`: Características gramaticales del contexto local.
4. `CountVectorsFeaturizer` (palabra): N-gramas de palabras (1–2).
5. `CountVectorsFeaturizer` (carácter): N-gramas de caracteres (1–4).
6. `DIETClassifier`: Clasificador dual de intenciones y entidades (150 épocas, dropout 0.2).
7. `EntitySynonymMapper`: Normalización de sinónimos de entidades.
8. `ResponseSelector`: Selección de respuestas tipo FAQ.
9. `FallbackClassifier`: Detección de baja confianza (umbral 0.7).

Justificación:

- La featurización a nivel de carácter es especialmente eficaz para manejar errores ortográficos (“Fundamnetos” se reconoce por similitud de caracteres con “Fundamentos”).
- El umbral de fallback de 0.7 equilibra la precisión (evitar respuestas incorrectas) con la cobertura (no rechazar demasiadas consultas legítimas).
- El `DIETClassifier` con 150 épocas y dropout de 0.2 proporciona un modelo robusto sin sobreajuste al conjunto de entrenamiento.

6.4.9. Hash SHA-256 para evitar reprocesamiento de documentos

Contexto: El pipeline de vectorización RAG procesa documentos PDF que pueden ser costosos de re-vectorizar (extracción de texto, chunking, generación de embeddings con API de pago por uso).

Decisión: Se almacena un **hash SHA-256** de cada PDF procesado en la tabla `planes_docentes`. Antes de procesar un documento, se compara el hash actual con el almacenado.

Justificación:

- Si el hash coincide, el documento no ha cambiado y se omite el reprocesamiento, ahorrando llamadas a la API de embeddings.
- Si el hash difiere, se eliminan los fragmentos anteriores y se reprocesa el documento completo, garantizando la consistencia de los datos.
- Este mecanismo es especialmente relevante dado el límite de 1000 embeddings por día del plan gratuito de Gemini.

6.5– Patrones arquitectónicos aplicados

El diseño del sistema incorpora varios patrones arquitectónicos reconocidos en la ingeniería del software:

6.5.1. Arquitectura por capas (*Layered Architecture*)

Como se ha descrito en la Sección 6.1, el sistema se organiza en tres capas con responsabilidades bien definidas. Cada capa solo se comunica con la inmediatamente inferior, lo que reduce el acoplamiento y facilita el reemplazo de componentes (por ejemplo, cambiar el frontend sin afectar la lógica de negocio).

6.5.2. Patrón *Action Server* (microservicio de lógica)

Rasa delega la ejecución de lógica de negocio en un servidor de acciones independiente. Este patrón es similar al patrón *Backend for Frontend* (BFF), donde un servicio intermedio adapta las respuestas de los servicios de datos al formato esperado por el cliente conversacional. La separación permite escalar el Action Server independientemente del motor Rasa.

6.5.3. Patrón *Strategy* para clientes LLM

Los clientes de Gemini (`gemini_client.py`) y Ollama (`ollama_client.py`) implementan una interfaz similar para la generación de texto. El Action Server puede alternar entre ambos proveedores en función de la disponibilidad, aplicando el patrón *Strategy* para desacoplar la lógica de negocio del proveedor concreto de LLM.

6.5.4. Patrón *Pipeline* para procesamiento RAG

El pipeline de vectorización sigue el patrón *Pipeline* (o *Pipes and Filters*), donde cada etapa transforma los datos y los pasa a la siguiente: extracción de PDF → chunking → generación de embeddings → inserción en base de datos. Cada etapa es independiente y puede modificarse sin afectar a las demás.

6.5.5. Patrón *Repository* para acceso a datos

El módulo `db.py` centraliza todas las operaciones contra la base de datos, proporcionando una abstracción que oculta los detalles de conexión (pool de conexiones `psycopg2`) y ejecución de consultas al resto del sistema. Esto facilita el testing unitario mediante mocks y permite cambiar la implementación de persistencia sin modificar la lógica de negocio.

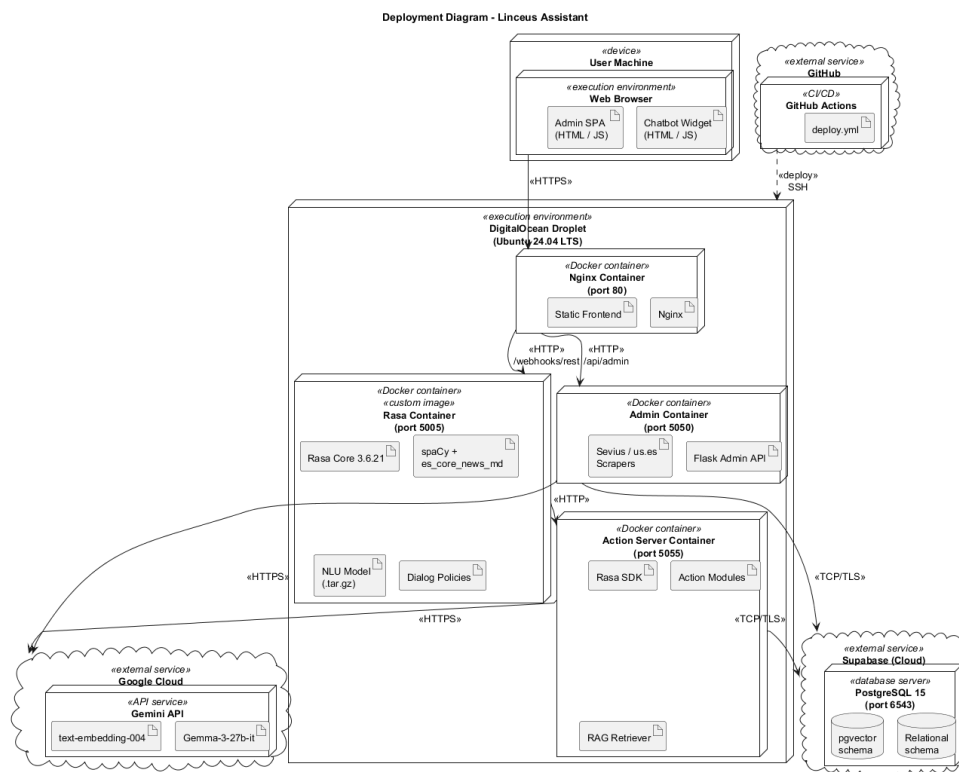


Figura 6.2: Diagrama de despliegue del sistema Linceus en DigitalOcean (Ubuntu 24.04 LTS) con el pipeline de despliegue automatizado vía GitHub Actions.

Marco tecnológico

Este capítulo describe el conjunto de tecnologías, frameworks y paradigmas que conforman la base técnica del proyecto LinceUS Assistant. Se complementa con el Capítulo 6, donde se detallan la arquitectura del sistema, las decisiones de diseño y los diagramas de componentes y despliegue. Aquí se justifica la elección de cada componente del stack tecnológico.

7.1– Tecnologías de procesamiento conversacional

7.1.1. Rasa Framework

Rasa es un framework de código abierto para la construcción de asistentes conversacionales basados en IA. A diferencia de soluciones comerciales como Dialogflow o Lex de AWS, Rasa ofrece:

- **Control total sobre el modelo:** Los datos de entrenamiento y los modelos permanecen bajo control del desarrollador
- **Personalización completa:** Políticas de diálogo y pipelines de NLU totalmente configurables
- **Ejecución local:** No depende de servicios externos para funcionar
- **Multilinguaje:** Soporte nativo para español mediante modelos de spaCy

Componentes de Rasa utilizados

Rasa NLU (Natural Language Understanding)

Rasa NLU se encarga de interpretar la entrada del usuario y extraer:

- **Intenciones (intents):** ¿Qué quiere hacer el usuario? (e.g., `consultar_horario`, `buscar_asignatura`)
- **Entidades (entities):** Información específica mencionada por el usuario (e.g., nombre de asignatura, curso, centro)

Rasa Core

Rasa Core gestiona el flujo de conversación mediante:

- **Stories:** Conversaciones de ejemplo que entrenan el modelo de diálogo
- **Rules:** Reglas determinísticas para situaciones específicas

- **Políticas:** Algoritmos de aprendizaje que deciden la siguiente acción del bot

Rasa Actions

Las acciones personalizadas permiten al bot ejecutar código Python para:

- Consultar la base de datos
- Generar respuestas dinámicas
- Llamar a APIs externas
- Procesar lógica de negocio compleja

7.1.2. Procesamiento de Lenguaje Natural

spaCy

spaCy proporciona capacidades avanzadas de NLP en español:

- **Tokenización:** Separación del texto en palabras y signos de puntuación
- **Lematización:** Reducción de palabras a su forma base
- **Análisis morfológico:** Identificación de categorías gramaticales
- **Vectores de palabras:** Representaciones numéricas del significado de las palabras

El modelo `es_core_news_md` incluye vocabulario específico del español y ha sido entrenado en corpus de textos en español.

Coincidencia difusa (Fuzzy Matching)

La librería `RapidFuzz` implementa algoritmos de distancia de cadenas que permiten:

- Tolerar errores ortográficos (e.g., "Fundamnetos de Programacion")
- Reconocer abreviaturas (e.g., "FP" por "Fundamentos de Programación")
- Manejar variaciones en la escritura (e.g., mayúsculas, tildes)

Se utiliza el algoritmo de *Levenshtein distance* con un umbral de similitud del 80 % para considerar coincidencias válidas.

7.2– Tecnologías de Inteligencia Artificial

7.2.1. Ollama y modelos de lenguaje grandes (LLMs)

Ollama es una plataforma que permite ejecutar modelos de lenguaje de gran tamaño localmente, sin necesidad de conexión a servicios cloud. En LinceUS Assistant se utiliza para dos propósitos principales:

Text-to-SQL

El sistema implementa un pipeline de generación de consultas SQL a partir de lenguaje natural:

1. El usuario formula una pregunta en lenguaje natural
2. Ollama (usando un modelo como Llama 3 o Mistral) genera la consulta SQL correspondiente
3. La consulta se valida y ejecuta contra la base de datos
4. Los resultados se procesan y formatean para presentarlos al usuario

Este enfoque permite consultas complejas sin necesidad de definir explícitamente todas las posibles preguntas.

Generación de respuestas contextualizadas

Para consultas que no se ajustan a patrones predefinidos, el modelo LLM genera respuestas naturales basándose en:

- El contexto de la conversación
- La información recuperada de la base de datos
- El conocimiento previo del modelo

7.2.2. Embeddings y búsqueda semántica

Para la recuperación de información de documentos (planes docentes, información de trámites), se utiliza:

- **Modelos de embeddings:** Transforman texto en vectores numéricos que capturan el significado semántico
- **pgvector:** Extensión de PostgreSQL que permite almacenar y buscar vectores eficientemente
- **Búsqueda de similitud:** Encuentra documentos relevantes usando distancia coseno entre vectores

Este enfoque implementa un sistema RAG (*Retrieval-Augmented Generation*) que mejora la precisión de las respuestas al combinar búsqueda semántica con generación de lenguaje.

7.3– Tecnologías de base de datos

7.3.1. PostgreSQL

PostgreSQL es el sistema de gestión de bases de datos relacional utilizado. Se ha elegido por:

- **Robustez:** Base de datos ACID con alta integridad de datos
- **Extensibilidad:** Soporte para tipos de datos personalizados y extensiones
- **Rendimiento:** Optimización de consultas complejas y soporte para índices avanzados
- **Código abierto:** Sin costes de licencia y comunidad activa

7.3.2. Supabase

Supabase proporciona una capa de servicios sobre PostgreSQL:

- **API REST automática:** Generación automática de endpoints para operaciones CRUD
- **Real-time:** Suscripciones a cambios en la base de datos
- **Autenticación:** Sistema de usuarios y permisos integrado
- **Almacenamiento:** Para archivos y documentos
- **Funciones Edge:** Ejecución de código serverless

La API REST de Supabase simplifica la integración con el backend de Python, eliminando la necesidad de escribir queries SQL manualmente para operaciones básicas.

7.3.3. pgvector

La extensión pgvector añade capacidades de búsqueda vectorial a PostgreSQL:

- Almacenamiento eficiente de vectores de alta dimensionalidad (hasta 16,000 dimensiones)
- Índices IVFFlat y HNSW para búsquedas rápidas de vecinos más cercanos
- Operadores de similitud (distancia euclidiana, producto punto, similitud coseno)

Esto permite implementar búsqueda semántica directamente en la base de datos, sin necesidad de sistemas externos como Pinecone o Weaviate.

7.4– Tecnologías de frontend

7.4.1. Node.js

El frontend del asistente está desarrollado usando Node.js, que proporciona:

- **JavaScript del lado del servidor:** Mismo lenguaje en frontend y backend de la interfaz
- **NPM:** Gestor de paquetes con millones de librerías disponibles
- **Rendimiento:** Motor V8 de alta velocidad

7.4.2. Framework web

Se utiliza un framework minimalista que permite:

- Servir la interfaz de chat web
- Gestionar la conexión WebSocket con el backend de Rasa
- Renderizar mensajes con formato enriquecido (tablas, enlaces, imágenes)

7.5– Herramientas de desarrollo

7.5.1. Control de versiones

Git es el sistema de control de versiones utilizado, con **GitHub** como plataforma de hosting. La estructura del repositorio incluye:

- `/actions`: Acciones personalizadas de Rasa
- `/data`: Datos de entrenamiento (intents, stories, rules)
- `/models`: Modelos entrenados de Rasa
- `/frontend`: Código de la interfaz web
- `/docs`: Documentación del proyecto

7.5.2. Gestión de dependencias

Las dependencias de Python se gestionan mediante:

- **requirements.txt**: Lista de todas las librerías necesarias con versiones específicas
- **Entorno virtual (venv)**: Aislamiento de dependencias del sistema

7.5.3. Variables de entorno

La librería `python-dotenv` gestiona la configuración mediante archivos `.env`:

- Credenciales de Supabase (URL, API key)
- Configuración de Ollama (URL del servidor, modelo a utilizar)
- Parámetros de configuración del sistema

Esto permite cambiar la configuración sin modificar el código y evita exponer credenciales en el repositorio público.

7.6– Consideraciones de seguridad

7.6.1. Protección de datos

- **Credenciales**: Nunca se almacenan en el código, siempre en variables de entorno
- **API keys**: Se utilizan claves específicas con permisos mínimos necesarios
- **Conexiones**: HTTPS para todas las comunicaciones con servicios externos

7.6.2. Validación de entrada

- Sanitización de consultas SQL para prevenir inyección SQL
- Validación de tipos de datos en entradas de usuario
- Limitación de longitud de mensajes

7.6.3. Ejecución local de modelos

El uso de Ollama para ejecutar modelos localmente aporta beneficios de privacidad:

- Las consultas de los usuarios no se envían a servicios externos
- No hay dependencia de APIs comerciales que puedan cambiar términos de uso
- Control total sobre los datos procesados

7.7– Stack tecnológico completo

La siguiente tabla resume todas las tecnologías empleadas en el proyecto:

Capa	Tecnología	Versión
Frontend	Node.js	18.x
Frontend	HTML/CSS/JavaScript	ES6+
Backend conversacional	Python	3.10
Backend conversacional	Rasa	3.6.21
Backend conversacional	Rasa SDK	3.6.2
Backend conversacional	spaCy	3.8.0
IA y LLMs	Ollama	latest
IA y LLMs	Modelos LLM	Llama 3 / Mistral
Base de datos	PostgreSQL	15.x
Base de datos	pgvector	0.5.0
Plataforma BD	Supabase	cloud
Librerías NLP	RapidFuzz	3.0.0
Librerías NLP	python-dotenv	1.0.0
Control versiones	Git & GitHub	2.x

Cuadro 7.1: Stack tecnológico completo del proyecto LinceUS Assistant.

Estructura del proyecto

Este capítulo describe la organización del repositorio LinceUS Assistant, detallando el propósito de cada directorio y fichero relevante. La estructura ha sido diseñada para separar claramente las responsabilidades entre los distintos módulos del sistema y facilitar el mantenimiento y la escalabilidad del proyecto.

8.1– Visión general del repositorio

El repositorio sigue una organización modular en la que cada directorio agrupa elementos con una responsabilidad concreta dentro del sistema. La separación entre la lógica conversacional, los datos de entrenamiento, la capa de acceso a datos, la interfaz de usuario y la documentación responde al principio de separación de responsabilidades (*Separation of Concerns*).

El árbol de directorios principal es el siguiente:

```
linceus-assistant/  
+-- actions/  
|   +-- __init__.py  
|   +-- actions.py  
|   +-- asignaturas.py  
|   +-- config.py  
|   +-- contexto.py  
|   +-- db.py  
|   +-- gemini_client.py  
|   +-- ollama_client.py  
|   \-- text_to_sql.py  
+-- data/  
|   +-- nlu/  
|   |   +-- asignaturas.yml  
|   |   +-- contexto.yml  
|   |   \-- general.yml  
|   +-- rules.yml  
|   \-- stories.yml  
+-- docs/  
|   +-- sprints/  
|   \-- other/  
+-- frontend/  
|   +-- chatbot-widget.js
```

```
|   +-- chatbot-icon.html
|   +-- pagina-principal.html
|   \-- pagina-principal.css
+-- memoria_tfg/
+-- tests/
|   \-- test_stories.yml
+-- config.yml
+-- domain.yml
+-- credentials.yml
+-- endpoints.yml
\-- requirements.txt
```

8.2– Directorio raíz

Los ficheros situados en la raíz del repositorio configuran el comportamiento global del sistema Rasa y la gestión de dependencias del proyecto.

- `config.yml`: Define el pipeline de procesamiento de lenguaje natural (NLU) y las políticas de diálogo de Rasa. Especifica los componentes utilizados para tokenización, extracción de características e identificación de intenciones y entidades.
- `domain.yml`: Declara el universo del asistente: intenciones, entidades, slots, acciones y respuestas predefinidas. Es el fichero central que Rasa utiliza para construir el modelo de diálogo.
- `credentials.yml`: Contiene las credenciales de conexión con los canales de comunicación externos (REST, Slack, etc.). Las claves sensibles se gestionan mediante variables de entorno para no exponerlas en el repositorio.
- `endpoints.yml`: Configura los endpoints de los servicios externos con los que se comunica Rasa, como el servidor de acciones personalizadas (*action server*).
- `requirements.txt`: Lista todas las dependencias de Python necesarias para ejecutar el proyecto, con versiones fijas para garantizar la reproducibilidad del entorno.

8.3– Directorio `actions/`

El directorio `actions/` contiene la lógica de negocio del asistente, implementada como acciones personalizadas de Rasa. Cuando el motor conversacional determina que debe ejecutar una acción, invoca el código Python correspondiente de este directorio a través del *action server*.

8.3.1. Organización de los módulos

Cada fichero encapsula una responsabilidad específica:

- `actions.py`: Punto de entrada principal. Define las acciones de Rasa (`ActionQueryAsignaturas`, `ActionFallback`, etc.) y orquesta las llamadas al resto de módulos según la intención detectada.
- `asignaturas.py`: Módulo especializado en la consulta de información sobre asignaturas. Implementa la lógica de búsqueda por nombre, curso, tipología y otras características, integrando coincidencia difusa (*fuzzy matching*) para tolerancia a errores ortográficos.

- `db.py`: Capa de acceso a datos. Centraliza todas las operaciones contra la base de datos Supabase (PostgreSQL), abstrayendo las consultas SQL del resto de la lógica de negocio.
- `config.py`: Carga y expone la configuración del sistema a partir de variables de entorno. Gestiona credenciales de Supabase, parámetros de Ollama y otros ajustes globales.
- `contexto.py`: Gestiona la memoria de contexto de la conversación. Mantiene información sobre la sesión del usuario (asignatura mencionada previamente, curso, etc.) para responder de forma coherente a preguntas de seguimiento.
- `text_to_sql.py`: Implementa el pipeline de generación de consultas SQL a partir de lenguaje natural mediante un modelo de lenguaje grande (LLM). Transforma preguntas del usuario en consultas SQL válidas para su ejecución contra la base de datos.
- `gemini_client.py`: Cliente de integración con la API de Gemini (Google). Gestiona las llamadas al modelo para la detección dinámica de intenciones y la generación de respuestas enriquecidas.
- `ollama_client.py`: Cliente de integración con Ollama para la ejecución local de modelos de lenguaje grande. Permite realizar inferencias sin depender de servicios externos en la nube.

8.3.2. Flujo de ejecución de una acción

Cuando el asistente recibe una consulta, el flujo típico dentro del directorio `actions/` es el siguiente:

1. `actions.py` recibe la petición del *action server* de Rasa.
2. Se extrae la intención y las entidades identificadas del tracker de Rasa.
3. Se consulta `contexto.py` para recuperar información de la sesión activa.
4. Se delega en el módulo específico (`asignaturas.py`, etc.) que ejecuta la consulta a través de `db.py`.
5. Si la consulta requiere generación de lenguaje natural, se invoca `gemini_client.py` u `ollama_client.py`.
6. La respuesta se devuelve a Rasa para ser enviada al usuario.

8.4– Directorio `data/`

El directorio `data/` contiene todos los datos de entrenamiento del modelo Rasa. Estos ficheros en formato YAML definen el comportamiento conversacional del asistente.

8.4.1. Subdirectorio `nlu/`

El subdirectorio `nlu/` agrupa los ficheros de entrenamiento del componente de comprensión del lenguaje natural (NLU), organizados temáticamente:

- `asignaturas.yml`: Ejemplos de entrenamiento para intenciones relacionadas con consultas sobre asignaturas: créditos, tipología, curso, duración, etc.
- `contexto.yml`: Ejemplos para intenciones de seguimiento contextual, como referencias anafóricas («¿y esa asignatura?», «¿cuántos créditos tiene?»).
- `general.yml`: Intenciones generales del asistente: saludo, despedida, afirmación, negación, solicitud de ayuda y gestión del fallback.

8.4.2. Ficheros de flujo conversacional

- `stories.yml`: Historias de entrenamiento que definen conversaciones de ejemplo. Rasa Core aprende de estas secuencias para predecir la siguiente acción del bot ante nuevas interacciones.
- `rules.yml`: Reglas determinísticas de alto nivel. A diferencia de las historias, las reglas se aplican siempre que se cumpla la condición, sin aprendizaje estadístico. Se utilizan para comportamientos críticos como el manejo del fallback o la finalización de formularios.

8.5– Directorio `frontend/`

El directorio `frontend/` contiene la interfaz web del asistente. Está diseñado como un widget embebible que puede integrarse en cualquier página web institucional.

- `chatbot-widget.js`: Componente principal del chat. Implementa la lógica de comunicación con el servidor Rasa mediante la API REST, gestiona el estado de la conversación en el cliente y renderiza los mensajes del asistente con soporte para texto enriquecido.
- `chatbot-icon.html`: Página de demostración que muestra el widget integrado con el icono flotante característico del asistente.
- `pagina-principal.html` y `pagina-principal.css`: Prototipo de la página principal de LinceUS, que sirve como entorno de prueba de la integración del chatbot en un contexto universitario real.

8.6– Directorio `data/` de Rasa vs. `docs/`

El directorio `docs/` almacena la documentación técnica del proyecto, organizada en sprints y documentos de referencia:

- `sprints/`: Carpetas por sprint con los documentos de planificación, decisiones técnicas y resultados de cada iteración del desarrollo.
- `other/`: Documentos de referencia transversales, como el esquema de la base de datos, investigaciones sobre tecnologías alternativas y planes de implementación.

8.7– Directorio `tests/`

El directorio `tests/` contiene los casos de prueba del sistema conversacional.

- `test_stories.yml`: Historias de prueba end-to-end siguiendo el formato nativo de Rasa para testing conversacional. Cada historia simula una conversación completa y verifica que el asistente responde con las acciones correctas en cada turno.

La estrategia de pruebas se complementa con scripts externos ubicados en `docs/other/`, que cubren pruebas de integración del pipeline Text-to-SQL y del módulo de contexto conversacional.

8.8– Separación de responsabilidades

La estructura del repositorio refleja una separación clara entre los distintos módulos del sistema, lo que aporta las siguientes ventajas:

- **Mantenibilidad:** Cada módulo puede modificarse de forma independiente sin afectar al resto del sistema, siempre que se respete la interfaz entre capas.
- **Testabilidad:** Los módulos de acceso a datos y de clientes de IA pueden probarse de forma aislada mediante mocks.
- **Escalabilidad:** Añadir soporte para nuevas épicas (profesores, horarios, trámites) implica crear nuevos ficheros en `actions/` y ampliar los datos en `data/nlu/`, sin modificar la arquitectura existente.
- **Colaboración:** La organización modular facilita el trabajo paralelo en distintas partes del sistema.

Estructura de la base de datos

Este capítulo describe el diseño y la estructura de la base de datos de LinceUS Assistant. La base de datos constituye el núcleo del sistema, almacenando toda la información académica necesaria para responder a las consultas de los estudiantes.

9.1– Diseño conceptual

El diseño de la base de datos se ha estructurado siguiendo el modelo relacional, organizando la información en entidades que representan los elementos principales del dominio universitario: universidades, centros, titulaciones, asignaturas, profesores, horarios y trámites administrativos.

9.1.1. Principios de diseño

El esquema de base de datos se ha diseñado siguiendo los siguientes principios:

1. **Normalización:** Las tablas están normalizadas hasta la tercera forma normal (3NF) para evitar redundancia y mantener la integridad referencial
2. **Escalabilidad:** El diseño permite añadir nuevas universidades, centros y titulaciones sin modificar la estructura
3. **Flexibilidad:** Campos de metadatos en formato JSONB permiten almacenar información variable sin alterar el esquema
4. **Búsqueda semántica:** Tablas de chunks con vectores de embeddings para búsqueda basada en similitud
5. **Trazabilidad:** Campos de timestamp (created_at, updated_at) en todas las tablas principales

9.2– Modelo entidad-relación

A continuación se detallan las principales entidades y sus relaciones del sistema.

9.3– Entidades principales

9.3.1. Universidades

La tabla `UNIVERSIDADES` almacena la información básica de cada universidad. Aunque el sistema está diseñado inicialmente para la Universidad de Sevilla, la estructura permite incorporar múltiples instituciones.

Campo	Tipo	Descripción
<code>id</code>	<code>uuid</code>	Identificador único (PK)
<code>codigo</code>	<code>varchar</code>	Código de la universidad (e.g., <code>ÜS</code>)
<code>nombre</code>	<code>varchar</code>	Nombre completo de la universidad
<code>nombre_corto</code>	<code>varchar</code>	Nombre abreviado
<code>dominio_web</code>	<code>varchar</code>	Dominio web oficial (e.g., <code>üs.es</code>)
<code>ciudad</code>	<code>varchar</code>	Ciudad donde se ubica
<code>activa</code>	<code>boolean</code>	Indica si está operativa
<code>created_at</code>	<code>timestampz</code>	Fecha de creación del registro
<code>updated_at</code>	<code>timestampz</code>	Fecha de última actualización

Cuadro 9.1: Estructura de la tabla `UNIVERSIDADES`

9.3.2. Centros

Los centros representan las escuelas y facultades de cada universidad (e.g., ETSII, Facultad de Química).

Campo	Tipo	Descripción
<code>id</code>	<code>uuid</code>	Identificador único (PK)
<code>universidad_id</code>	<code>uuid</code>	Referencia a <code>UNIVERSIDADES</code> (FK)
<code>codigo</code>	<code>varchar</code>	Código del centro (e.g., <code>.^{ET}SII</code>)
<code>nombre</code>	<code>varchar</code>	Nombre completo del centro
<code>nombre_corto</code>	<code>varchar</code>	Nombre abreviado
<code>direccion</code>	<code>varchar</code>	Dirección física del centro
<code>telefono</code>	<code>varchar</code>	Teléfono de contacto
<code>email</code>	<code>varchar</code>	Email de contacto
<code>web</code>	<code>varchar</code>	URL del sitio web del centro
<code>activo</code>	<code>boolean</code>	Indica si está operativo

Cuadro 9.2: Estructura de la tabla `CENTROS`

9.3.3. Departamentos

Los departamentos son unidades organizativas que agrupan profesores e imparten asignaturas.

9.3.4. Titulaciones

Las titulaciones representan los grados, másteres y doctorados ofrecidos por los centros.

9.3.5. Asignaturas

Las asignaturas son las materias que componen cada titulación.

El campo `nombre_normalizado` facilita la búsqueda tolerante a errores, mientras que `palabras_clave` permite búsquedas por términos relacionados.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
universidad_id	uuid	Referencia a UNIVERSIDADES (FK)
codigo	varchar	Código del departamento
nombre	varchar	Nombre completo
siglas	varchar	Siglas del departamento (e.g., "LSI")
email	varchar	Email de contacto
web	varchar	URL del sitio web
activo	boolean	Indica si está operativo

Cuadro 9.3: Estructura de la tabla DEPARTAMENTOS

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
centro_id	uuid	Referencia a CENTROS (FK)
codigo	varchar	Código interno (e.g., "GII-IS")
codigo_oficial	varchar	Código RUCT oficial
nombre	varchar	Nombre completo de la titulación
nombre_corto	varchar	Nombre abreviado
tipo	varchar	GRADO, MASTER o DOCTORADO
plan_estudios_anio	int	Año del plan de estudios vigente
creditos_totales	int	Créditos ECTS totales
duracion_anios	int	Duración en años
requisito_idioma	varchar	Nivel de idioma requerido (e.g., "B1")
activa	boolean	Indica si está vigente

Cuadro 9.4: Estructura de la tabla TITULACIONES

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
titulacion_id	uuid	Referencia a TITULACIONES (FK)
departamento_id	uuid	Referencia a DEPARTAMENTOS (FK)
codigo	varchar	Código de la asignatura
nombre	varchar	Nombre oficial de la asignatura
curso	int	Curso en el que se imparte (1-4)
creditos	decimal	Créditos ECTS
duracion	varchar	A (anual), C1 (1er cuatr.), C2 (2º cuatr.)
tipologia	varchar	TRONCAL, OBLIGATORIA, OP-TATIVA
es_formacion_basica	boolean	Si es de formación básica
es_optativa	boolean	Si es optativa
nombre_normalizado	varchar	Nombre sin tildes ni mayúsculas
palabras_clave	text[]	Array de palabras clave
activa	boolean	Indica si está vigente

Cuadro 9.5: Estructura de la tabla ASIGNATURAS

9.3.6. Profesores

La tabla de profesores almacena la información del personal docente.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
departamento_id	uuid	Referencia a DEPARTAMENTOS (FK)
nombre	varchar	Nombre del profesor
apellidos	varchar	Apellidos del profesor
nombre_completo	varchar	Nombre completo (generado)
nombre_normalizado	varchar	Versión normalizada para búsqueda
email	varchar	Email de contacto
telefono	varchar	Teléfono de contacto
despacho	varchar	Número de despacho (e.g., "F1.45")
edificio	varchar	Edificio donde se ubica
planta	varchar	Planta del despacho
web_personal	varchar	URL de página personal
orcid	varchar	Identificador ORCID
activo	boolean	Indica si está en activo

Cuadro 9.6: Estructura de la tabla PROFESORES

9.3.7. Tutorías

Las tutorías representan los horarios de atención a estudiantes por parte de los profesores.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
profesor_id	uuid	Referencia a PROFESORES (FK)
día_semana	int	Día de la semana (1=lunes, 5=viernes)
hora_inicio	time	Hora de inicio de la tutoría
hora_fin	time	Hora de fin de la tutoría
ubicacion	varchar	Lugar donde se realiza
modalidad	varchar	presencial, online o mixta
enlace_online	varchar	URL para tutorías online
curso_academico	varchar	Curso académico (e.g., "2025-26")
cuatrimestre	varchar	C1, C2 o A (anual)
notas	text	Observaciones adicionales
activa	boolean	Indica si está vigente

Cuadro 9.7: Estructura de la tabla TUTORIAS

9.3.8. Horarios

Los horarios relacionan grupos de clase con aulas, profesores y franjas horarias.

9.4– Tablas de vectorización (RAG)

Para implementar el sistema RAG (*Retrieval-Augmented Generation*), se han diseñado tablas especializadas que almacenan fragmentos de documentos junto con sus representaciones vectoriales.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
grupo_id	uuid	Referencia a GRUPOS_CLASE (FK)
aula_id	uuid	Referencia a AULAS (FK)
profesor_id	uuid	Referencia a PROFESORES (FK)
día_semana	int	Día de la semana (1-5)
hora_inicio	time	Hora de inicio de la clase
hora_fin	time	Hora de fin de la clase
fecha_inicio	date	Fecha de inicio del periodo
fecha_fin	date	Fecha de fin del periodo
notas	text	Observaciones
activo	boolean	Indica si está vigente

Cuadro 9.8: Estructura de la tabla HORARIOS

9.4.1. Planes docentes

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
asignatura_id	uuid	Referencia a ASIGNATURAS (FK)
curso_academico	varchar	Curso académico (e.g., "2025-26")
grupo	varchar	Grupo al que corresponde
url_documento	varchar	URL del PDF original
hash_documento	varchar	Hash SHA256 del documento
fecha_documento	date	Fecha del documento
coordinador_nombre	varchar	Nombre del coordinador
estado_rag	varchar	pendiente, procesando, completado, error
fecha_procesamiento	timestampz	Fecha de procesamiento
error_procesamiento	text	Mensaje de error si lo hay
updated_at	timestampz	Última actualización

Cuadro 9.9: Estructura de la tabla PLANES_DOCENTES

9.4.2. Chunks de planes docentes

El campo `embedding` utiliza el tipo `vector` proporcionado por la extensión `pgvector`. Los vectores se generan usando modelos de embeddings especializados y permiten búsquedas de similitud semántica mediante consultas como:

```
SELECT contenido, embedding <=> query_vector AS distance
FROM planes_docentes_chunks
ORDER BY distance
LIMIT 5;
```

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
plan_docente_id	uuid	Referencia a PLANES_DOCENTES (FK)
contenido	text	Texto del fragmento
embedding	vector	Vector de 768 dimensiones
seccion	varchar	Sección del documento (e.g., "Evaluación")
subseccion	varchar	Subsección si aplica
numero_pagina	int	Número de página del PDF
orden_chunk	int	Orden del chunk en el documento
metadata	jsonb	Metadatos adicionales en formato JSON
updated_at	timestampz	Última actualización

Cuadro 9.10: Estructura de la tabla PLANES_DOCENTES_CHUNKS

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
codigo	varchar	Código de la categoría (e.g., "matricula")
nombre	varchar	Nombre de la categoría
descripcion	text	Descripción detallada
icono	varchar	Nombre del icono a mostrar
orden	int	Orden de presentación
activa	boolean	Indica si está activa

Cuadro 9.11: Estructura de la tabla CATEGORIAS_TRAMITES

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
categoria_id	uuid	Referencia a CATEGORIAS_TRAMITES (FK)
universidad_id	uuid	Referencia a UNIVERSIDADES (FK)
codigo	varchar	Código del trámite
nombre	varchar	Nombre del trámite
descripcion	text	Descripción detallada
url_oficial	varchar	URL oficial del trámite
url_formulario	varchar	URL del formulario si existe
palabras_clave	text[]	Palabras clave para búsqueda
estado_rag	varchar	Estado del procesamiento RAG
fecha_procesamiento	timestampz	Fecha de procesamiento
activo	boolean	Indica si está vigente

Cuadro 9.12: Estructura de la tabla TRAMITES

9.5– Tablas de trámites administrativos

9.5.1. Categorías de trámites

9.5.2. Trámites

9.6– Relaciones entre entidades

Las principales relaciones del modelo son:

- Una **universidad** tiene múltiples **centros** y **departamentos**
- Un **centro** ofrece múltiples **titulaciones** y tiene múltiples **aulas**
- Una **titulación** contiene múltiples **asignaturas**
- Una **asignatura** es impartida por un **departamento** y tiene múltiples **planes docentes** y **grupos de clase**
- Un **departamento** agrupa a múltiples **profesores**
- Un **profesor** imparte múltiples **asignaturas**, ofrece **tutorías** y está asignado a **horarios**
- Un **grupo de clase** tiene un conjunto de **horarios** asociados
- Un **horario** relaciona un **grupo**, un **aula** y un **profesor**

9.7– Índices y optimización

Para garantizar un rendimiento óptimo en las consultas, se han creado los siguientes índices:

- **Índices B-tree**: En todas las claves primarias y foráneas
- **Índices de texto**: En campos `nombre_normalizado` para búsquedas rápidas
- **Índices GIN**: En campos de array (`palabras_clave`) y JSONB (`metadata`)
- **Índices IVFFlat**: En campos `embedding` para búsquedas de similitud vectorial

Ejemplo de creación de índice vectorial:

```
CREATE INDEX idx_planes_chunks_embedding
ON planes_docentes_chunks
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

9.8– Integridad referencial

Todas las relaciones entre tablas están protegidas por restricciones de clave foránea con políticas de borrado apropiadas:

- **ON DELETE CASCADE**: En relaciones donde el hijo no tiene sentido sin el padre (e.g., `chunks` sin plan docente)
- **ON DELETE RESTRICT**: En relaciones donde se debe prevenir el borrado accidental (e.g., no borrar una universidad con centros asociados)
- **ON DELETE SET NULL**: En relaciones opcionales donde el padre puede desaparecer sin eliminar al hijo

9.9– Estrategia de datos de prueba

Para el desarrollo y pruebas del sistema se han creado datos de prueba que incluyen:

- Universidad de Sevilla con varios centros (ETSII, Facultad de Informática)
- Titulaciones de Ingeniería Informática (IS, IC, TI)
- Conjunto representativo de asignaturas de cada titulación
- Profesores del Departamento de Lenguajes y Sistemas Informáticos
- Horarios del curso 2025-2026
- Planes docentes y trámites administrativos más consultados

Los datos de prueba permiten validar el funcionamiento del sistema sin necesidad de tener acceso a datos reales completos.

Referencias

- Autores, V. (2014). *Escuela Técnica Superior de Ingeniería Informática*. ETSII. Descargado de <http://www.informatica.us.es> (fecha de consulta: 24 de Noviembre de 2014)
- Bezoz, J. (2007). The titlesec and titletoc packages. *TexEmplares*, 8, 283–298.
- de Sousa, J. M. (2004). *Ortografía y ortotipografía del español actual*. Trea.